

UNIVERSITETET I OSLO

Nevrale Nettverk

Kai Olav Ellefsen
*Gruppe for Robotikk og Intelligente
Systemer (ROBIN)*

IN1160 – Introduksjon til
maskinlæring (Vår 2026)



Praktisk info

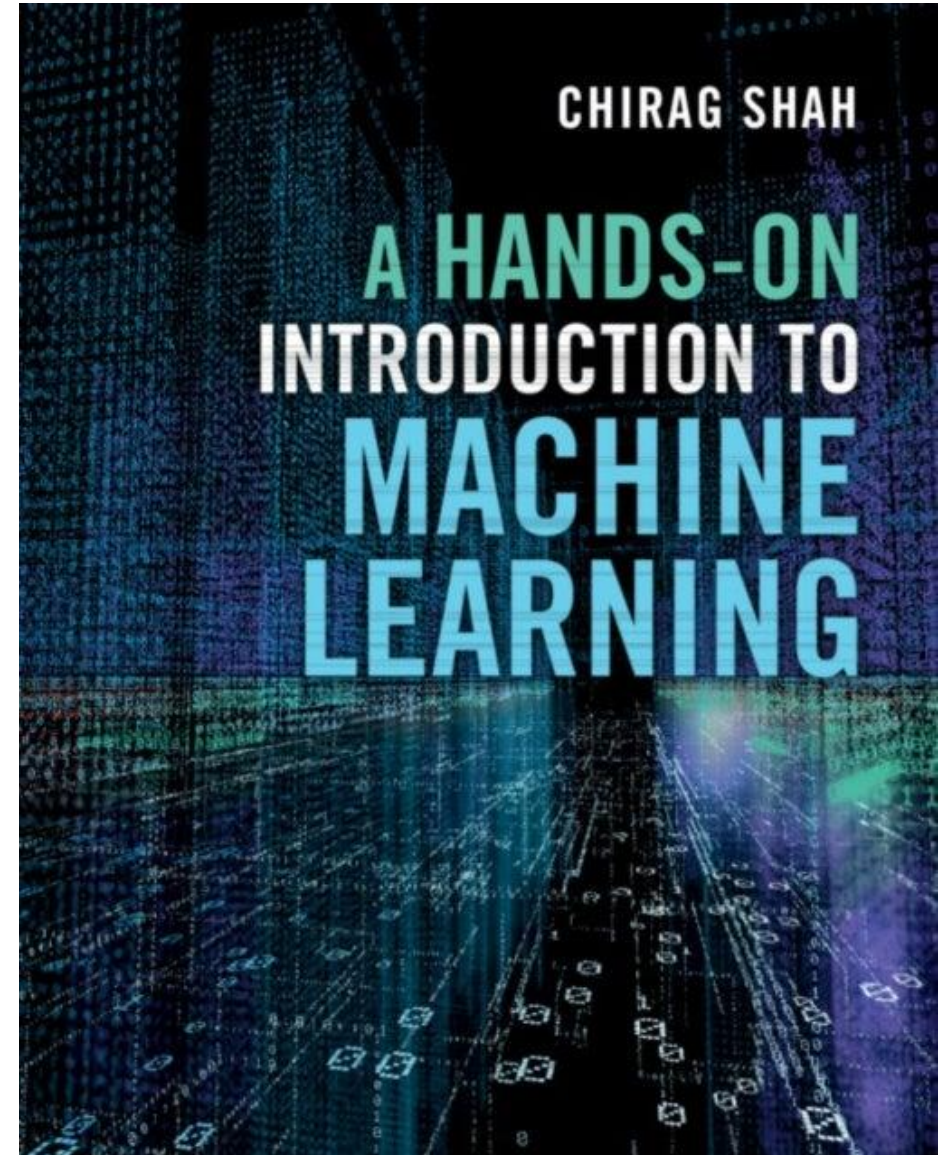
- Neste uke tar vi en [pause](#) i forelesningene
- Men seminarer og labber fortsetter
- Bruk tiden til å repetere, og å starte på Oblig 2b, om nevrale nettverk.
- Svar også gjerne på vårt skjema for midtveisevaluering (TODO: Link)

Nevrale Nettverk – litt kontekst

- **Nevrale Nettverk er teknologien bak Dyp Læring** – som står bak de største gjennombruddene i Maskinlæring de siste 10-15 år
 - **Hva er forskjellen mellom et Nevralt Nettverk og Dyp Læring?**
- **Nevrale Nettverk er veldig fleksible** – brukes til både veiledet, ikke-veiledet og forsterkende læring. Brukes til både klassifikasjon og regresjon.
 - **Arkitektur, treningsmetode, inndata og utdata** vil såklart variere mellom disse ulike anvendelsene

Pensum idag

- Pensum: Shah, kap 9.1 og 9.2
- Ikke pensum: Kap 9.3 – 9.6, som omhandler mer avanserte typer nevrale nettverk. Vi kommer litt inn på disse senere i dette kurset. Andre ML-kurs på IFI går mye mer i dybden.



A detailed illustration of a neural network. The background is filled with a dense web of neurons. The neurons have star-shaped cell bodies (soma) with multiple branching processes (dendrites and axons) extending outwards. The colors of the neurons vary, including shades of blue, green, yellow, orange, red, and purple, creating a vibrant and complex visual. The text is centered over this background.

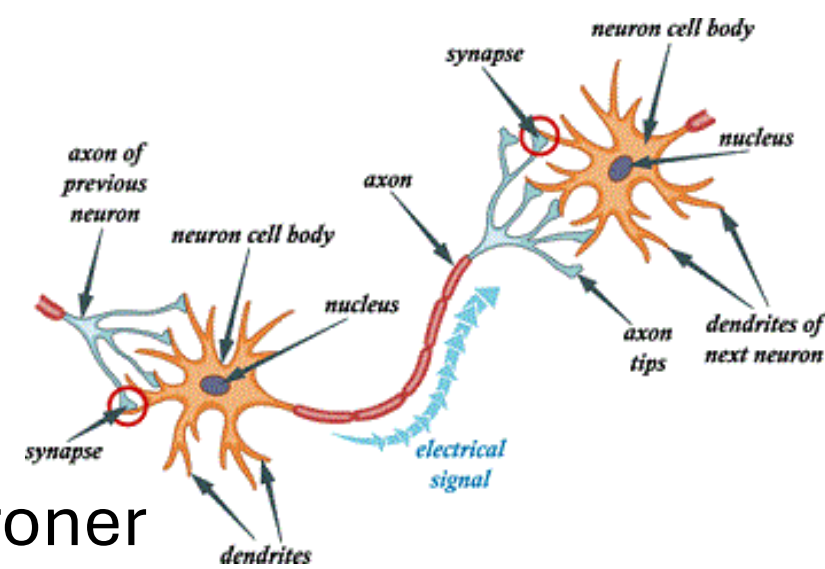
1. Å etterlikne menneskehjernen

Hjernen din

- **Hjernen er mer effektiv** (den lærer mer fra mindre data) enn de kraftigste ML-algoritmer. Naturlig å se til den for inspirasjon
- Hjernen din har ca **100 milliarder nerveceller – «nevroner»**
- Hvert nevron er koblet med opptil 10.000 andre nevroner
- Disse **sammenkoblingene (synapser)** bestemmer alt vi gjør – og de oppdateres/læres gjennom livet.



Biologiske Nevrale Nettverk



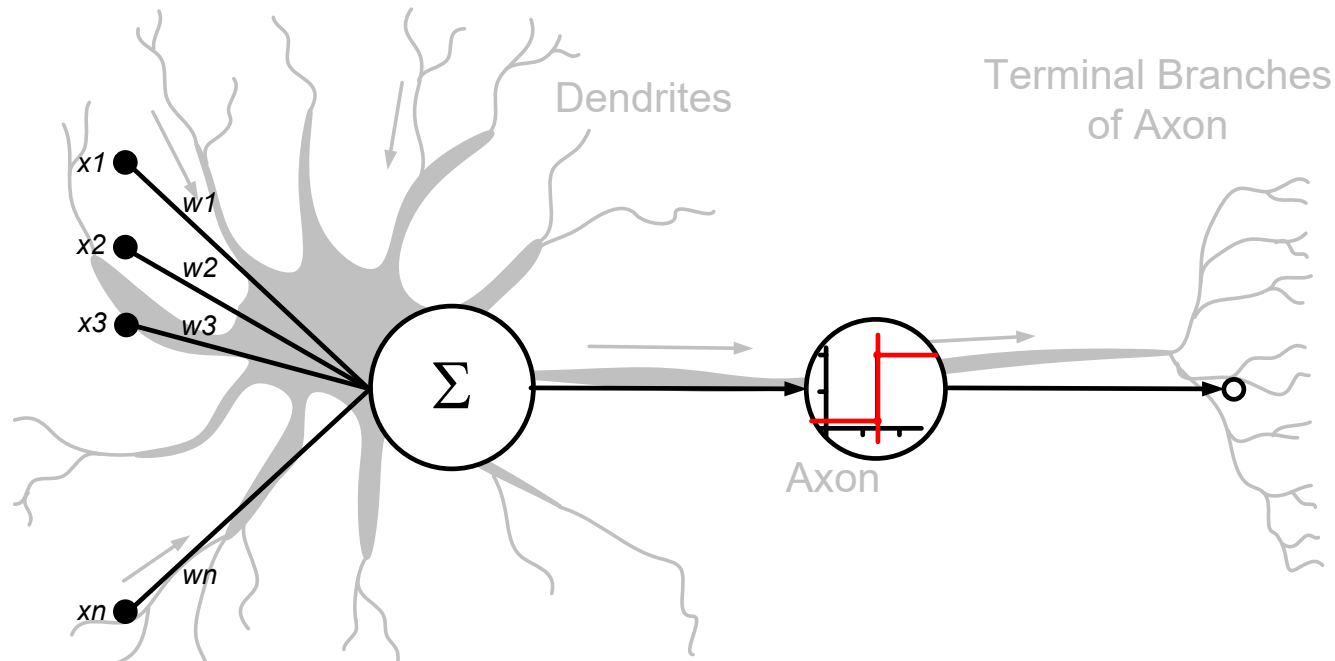
- **Elektriske signaler** beveger seg gjennom nevroner
- Nevroner er koblet sammen i **synapser**
- Om et sterkt signal når en synapse, sender synapsen ut et **kjemisk signal**
- Disse kjemikaliene kan **aktivere nabo-nevroner**. Blir et nevron aktivert sterkt (gjerne av flere synapser samtidig), sender det ut sitt eget elektriske signal
- **Læring skjer ved at synapser endrer seg** – de vokser eller krymper, sensitiviteten deres øker eller minker, etc.

Matematiske modeller av nevrale nettverk

- De første slike modellene kom på 40-tallet
- Foreslo **hvordan informasjonsflyt og læring i nevrale nettverk kan regnes på**
- **McCulloch&Pitts (1943): Første matematiske modell** av et nevralt nettverk
- Noen av ideene fra denne modellen er fortsatt gjeldende i dagens dype nevrale nettverk

Mcculloch & Pitts nevroner

- Svært **forenklet modell av et nevron** som en **binær klassifikator**
- Det kunstige nevronet **beregner summen av vektet inndata**
 - Vi kan tenke oss at en **vekt** indikerer styrken på en **synapse**
 - Er summen over en terskelverdi, «fyrer» nevronet (gir ut en 1)
 - Ellers er det stille (gir ut en 0)

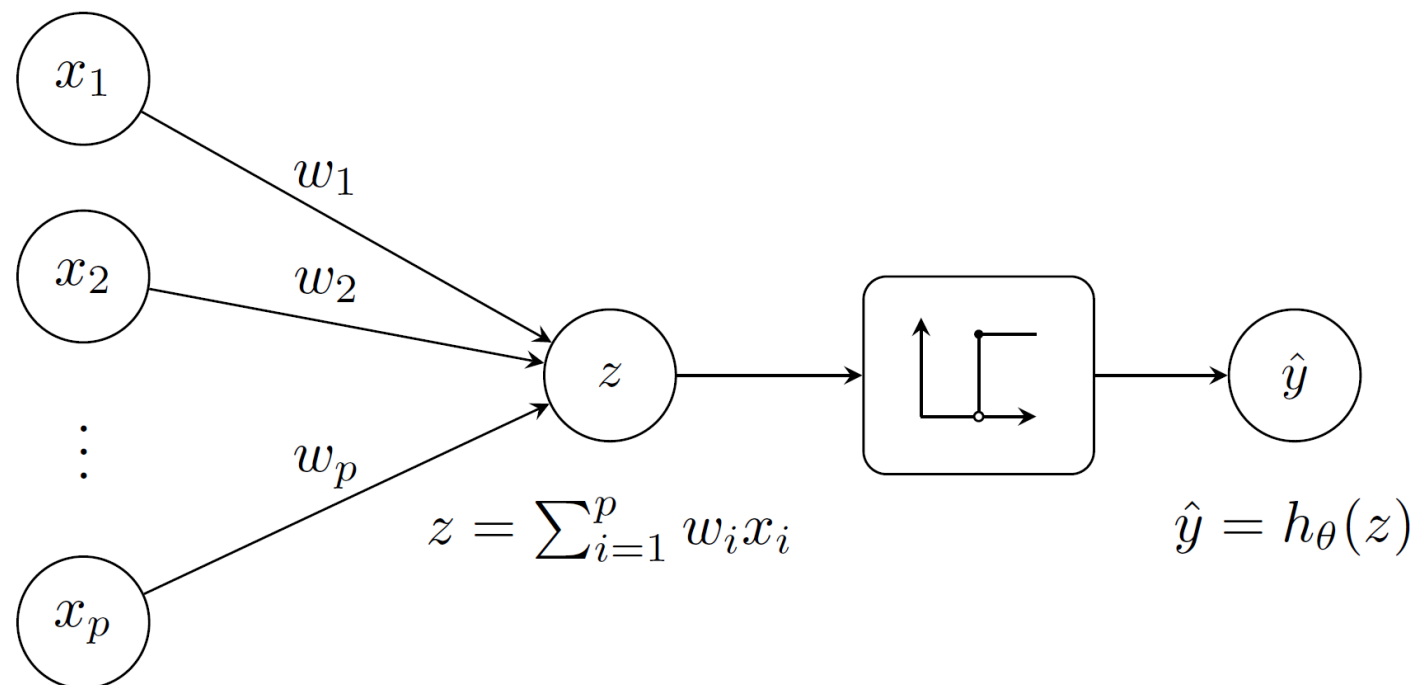
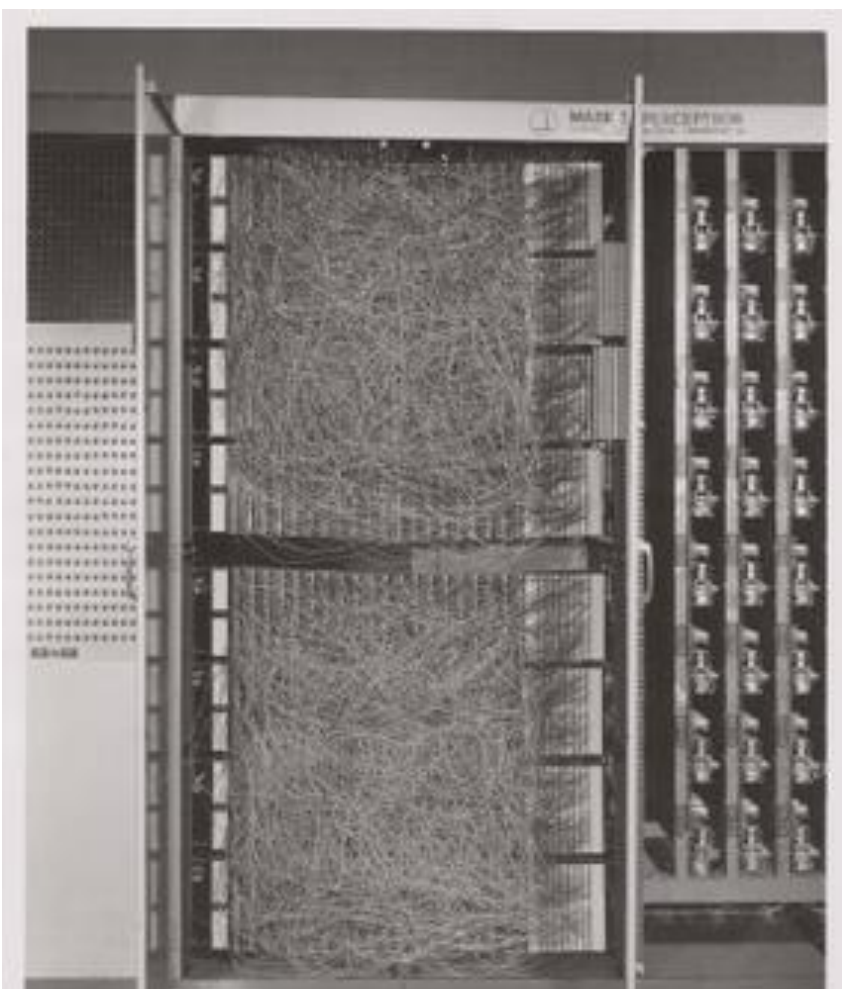




2. Perseptronet

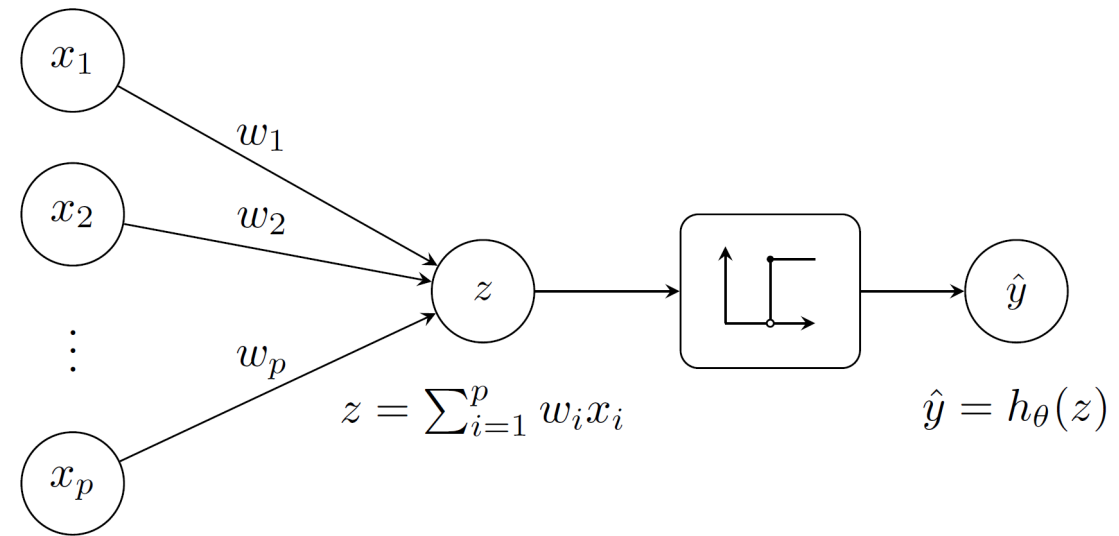
Gjennombruddet for nevrale nettverk som maskinlæringsalgoritme

1958: Frank Rosenblatt. Perceptron.



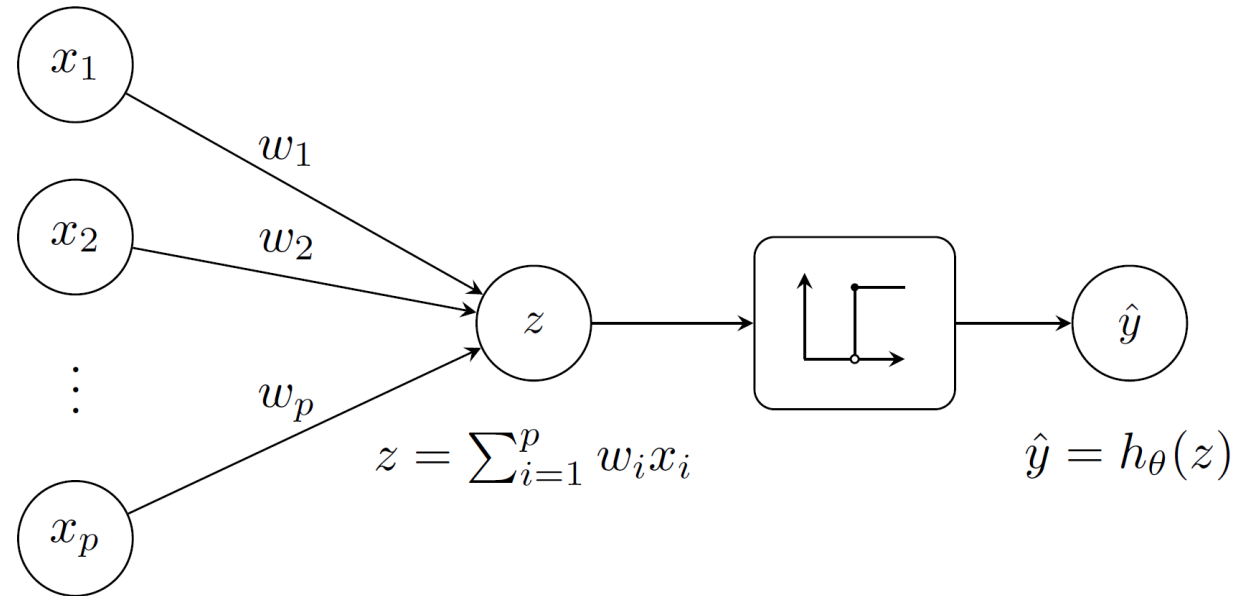
Ettlags-perseptronet

- Et **forovermatet («feed forward»)** nevralt nettverk (signalene går bare i én retning) for **binær klassifisering**
- Beregner **summen av vektet inndata**, og sender denne inn i en **aktiveringsfunksjon**.
- **Aktiveringsfunksjonen** er en enkel **terskelfunksjon**
- **Læring** skjer ved å justere **vektene**
- **Ut-verdien**: 1 hvis terskelen er overskredet, ellers 0.



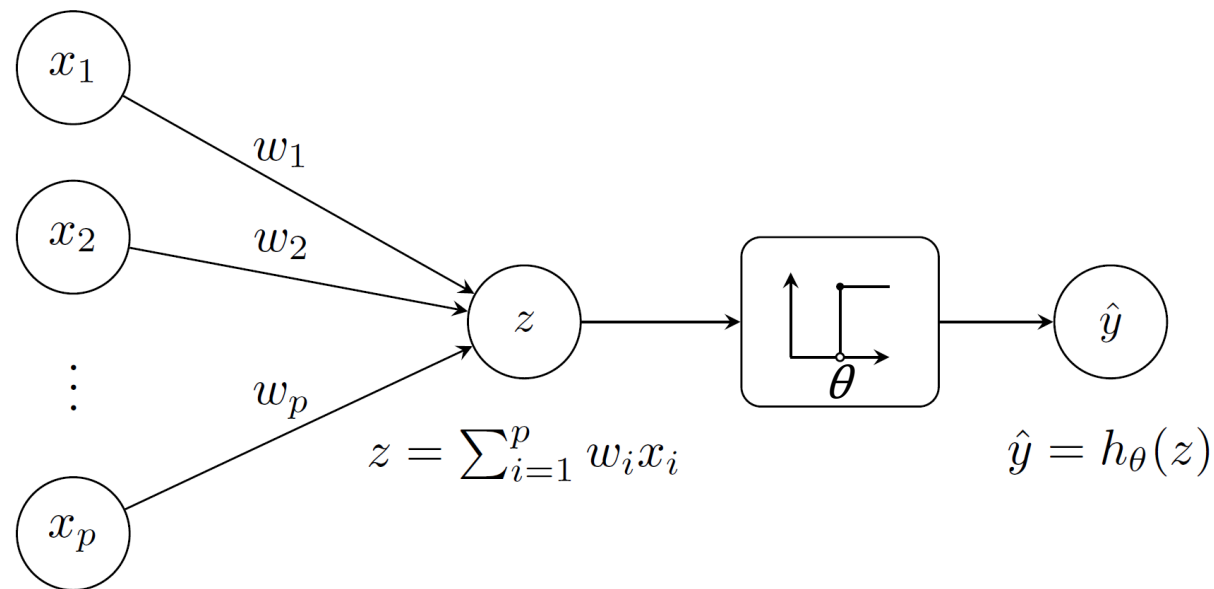
Ettlags-perseptronet - forståelse

- Hvorfor kaller vi utdata for $\hat{y}$?
Hva betyr «hatten»?
- Hvorfor har
aktiveringsfunksjonen den
merkelige θ -variabelen?



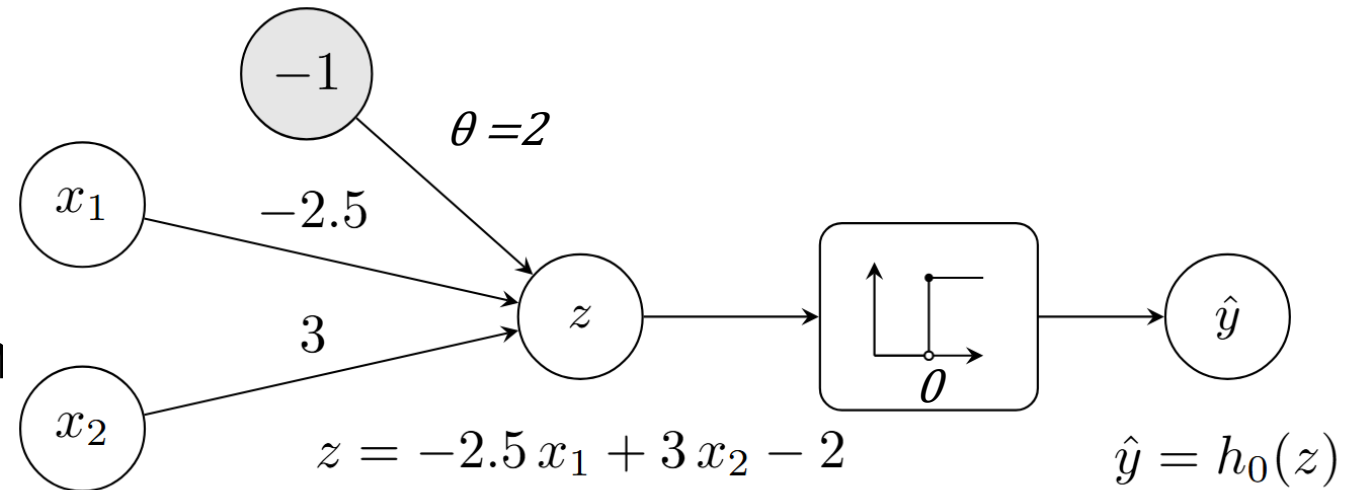
Ettlags-perseptronet - forståelse

- Hvorfor har aktiveringsfunksjonen den merkelige θ -variabelen?
 - θ indikerer her en **lærbar parameter**: Grensen for at perseptronet «fyrer» justeres under læring.
- Hvorfor kaller vi output for $\hat{y}$? Hva betyr «hatten»?
 - **Hatten** skiller den *predikerte* utverdien her fra den *sanne verdien* y



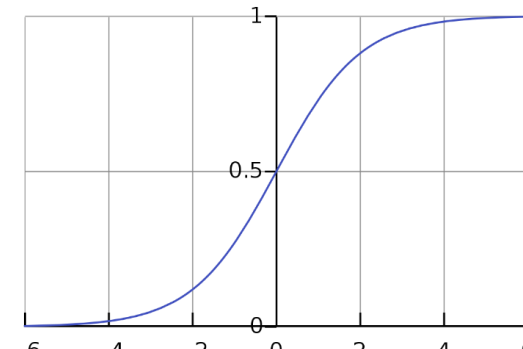
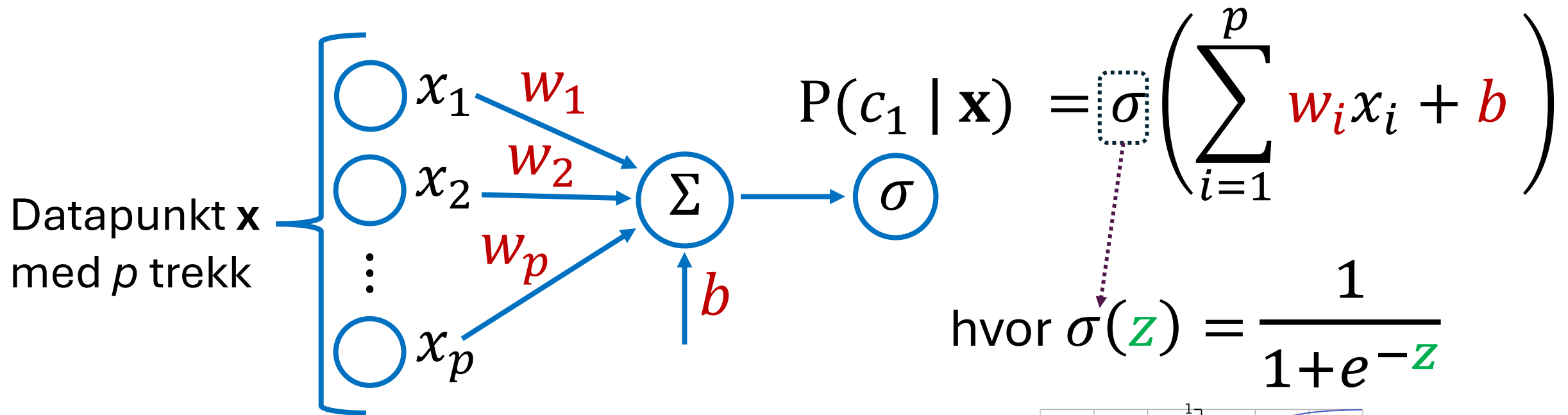
θ kalles også ofte bias

- Vi gir en **ekstra input (bias-input)** som alltid er -1
- Denne vektes med den lærte vekten θ
- Dermed kan **grensen for aktivering læres** akkurat som alle de andre vektene.
- Vi tegner ikke bias-inputen på de neste figurene, men i praksis er den alltid med



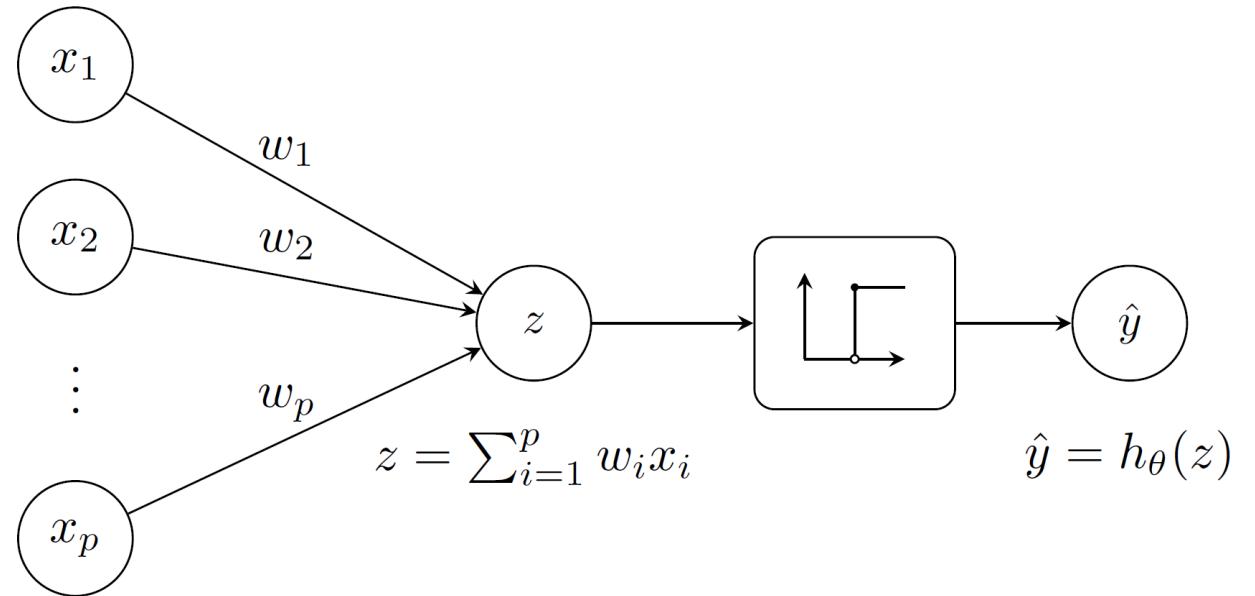
Påminner fra sist uke:

Logistisk regresjon med 2 klasser

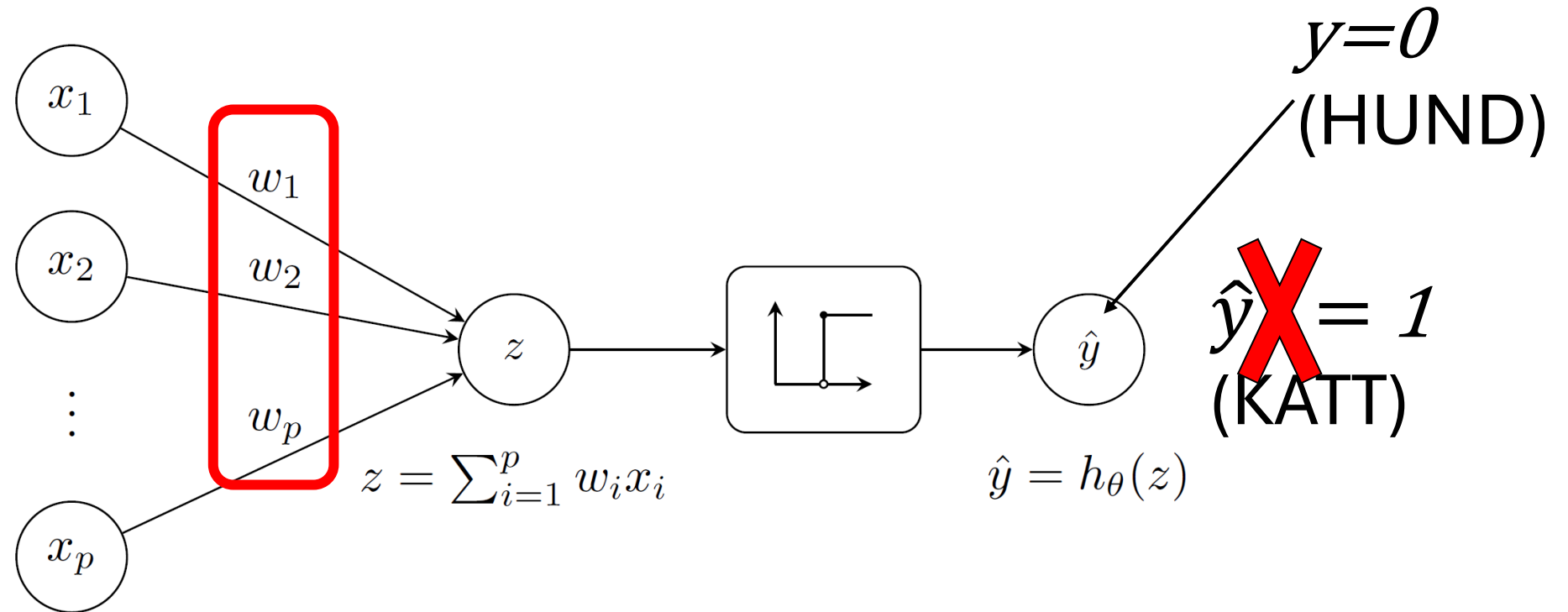


Perseptronet - læring

- Perseptronet trenes vha. **veiledet læring**
- Inndataene (x) er **trekk**, **utverdien** (1 eller 0) er en **binær klassifisering**
- Hvordan kan vi **trene perseptronet basert på eksempler** på trekk og klassifiseringer?

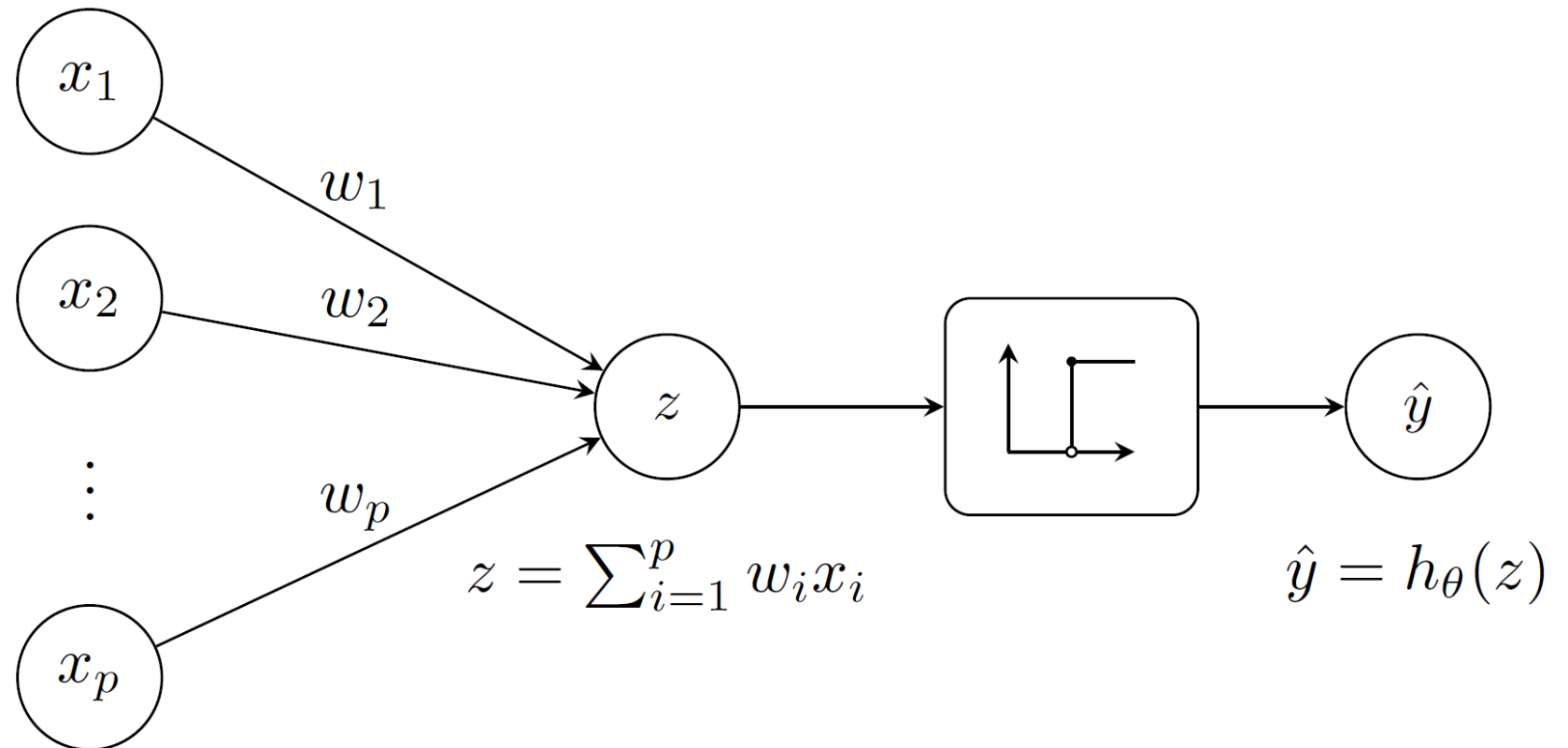


Eksempel: Lære å gjenkjenne en hund



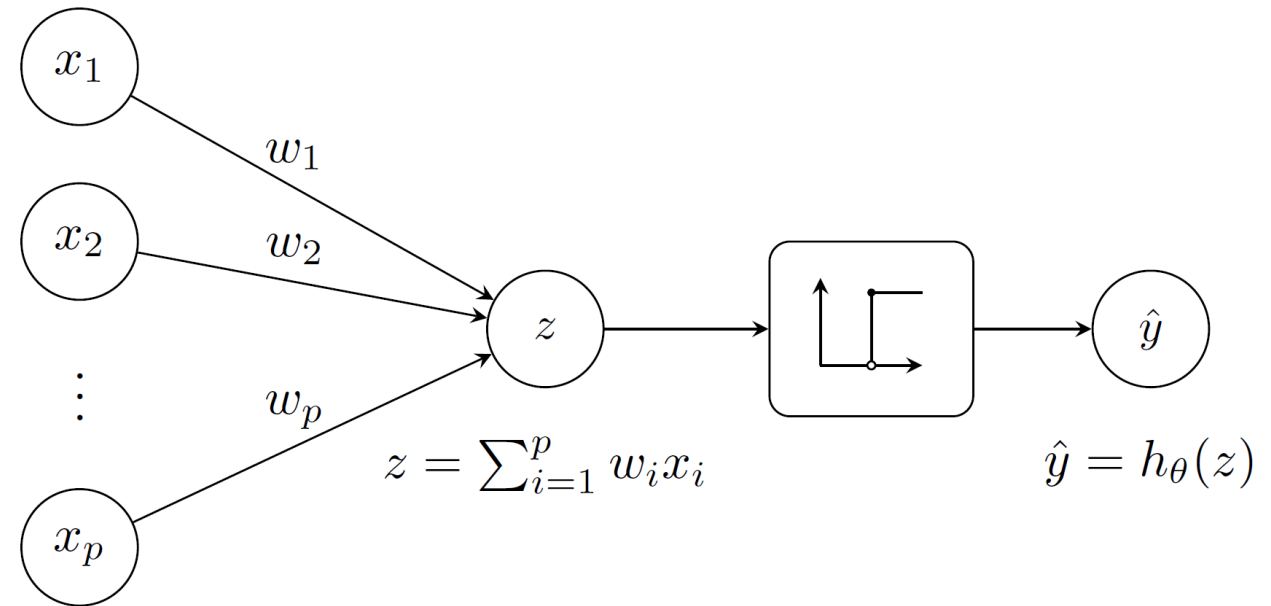
Perseptronet - læring

- Hvordan skal vi justere vektene?



Perseptron-læringsregelen

- Juster hver vekt proporsjonalt med **forskjellen på korrekt ut-verdi (y) og predikert ut-verdi ($\hat{y}$)**
- $\Delta w_i = \eta(y - \hat{y})x_i$
 - η er læringsraten (mellom 0 og 1)
 - y er korrekt ut-verdi («ground truth»)
 - $\hat{y}$ er predikert utverdi
 - x_i er inn-verdi



Enkelt eksempel

Vi skal trene opp perseptronet til høyre. Vektene er foreløpig tilfeldige.

Vi har 2 trekk som vi måler for hvert dyr. For eksempel tungelengde og ørelengde

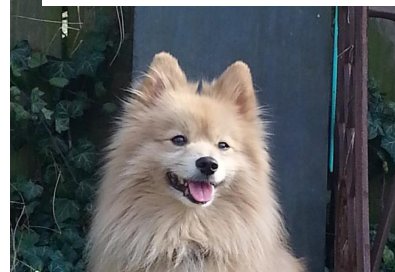
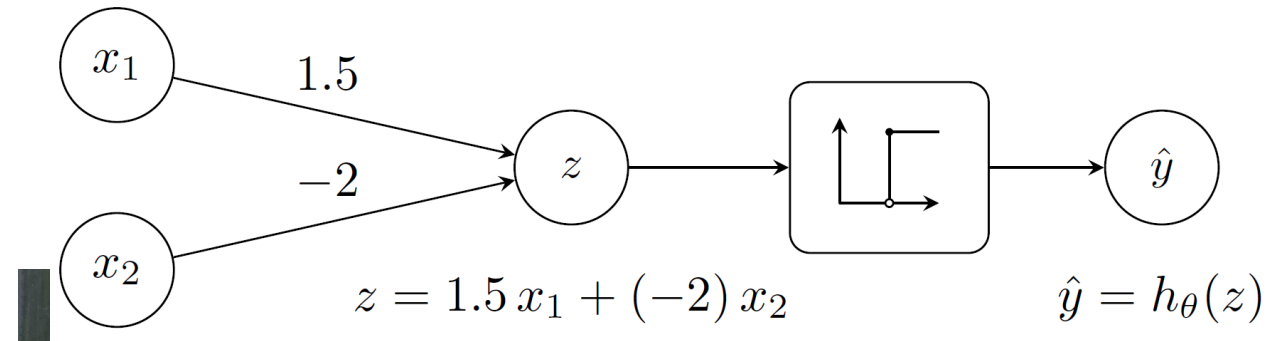
For den første hunden vi måler er disse 3cm og 2cm ($x_1 = 3$, $x_2 = 2$).

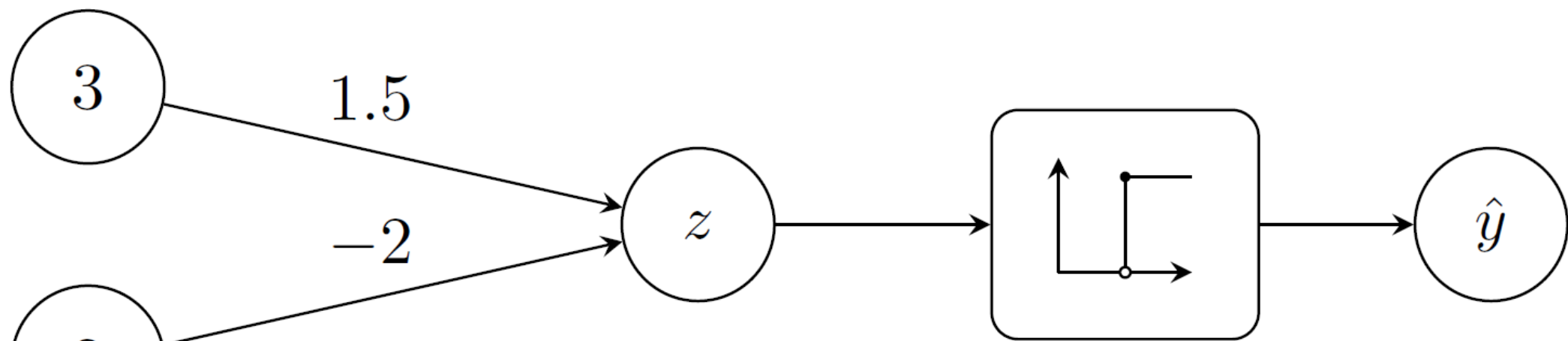
Korrekt utdata $y = 0$ (0 for hund, 1 for katt)

Terskelverdi/bias $\theta: 0$ (anta den er fast, ikke lærbar nå for eksempelets skyld)

Læringsrate $\eta: 0.1$

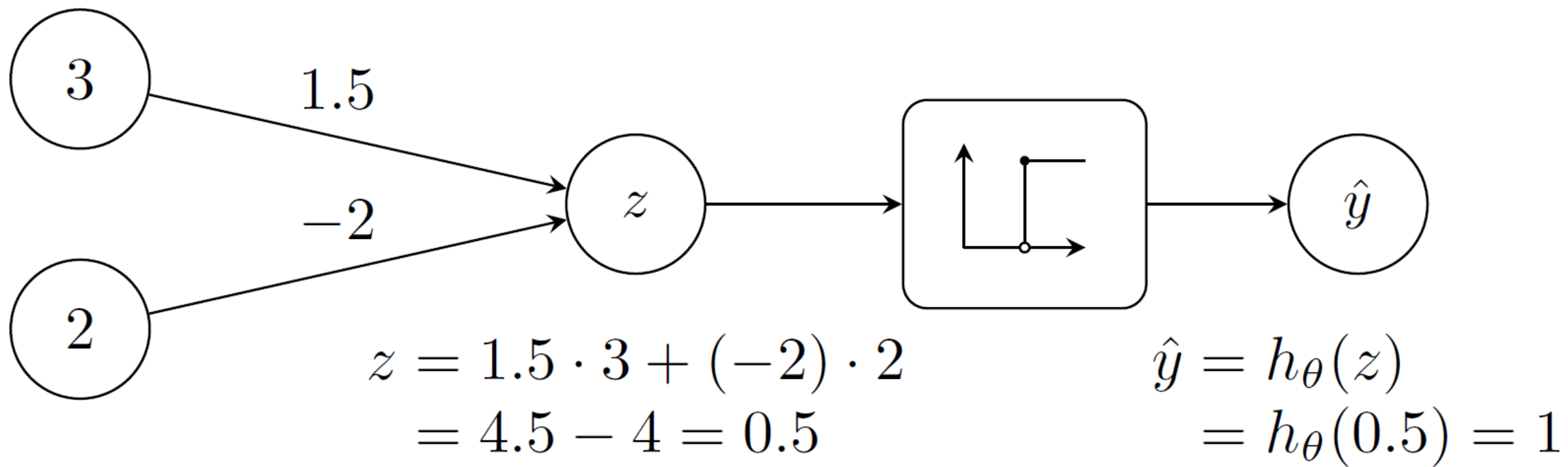
Hva vil perseptronet predikere?





$$\begin{aligned} z &= 1.5 \cdot 3 + (-2) \cdot 2 \\ &= 4.5 - 4 = 0.5 \end{aligned}$$

(0 = hund, 1 = katt)



(0 = hund, 1 = katt)

$$x_1 = 3, x_2 = 2$$

$y = 0$ (0 for hund, 1 for katt)

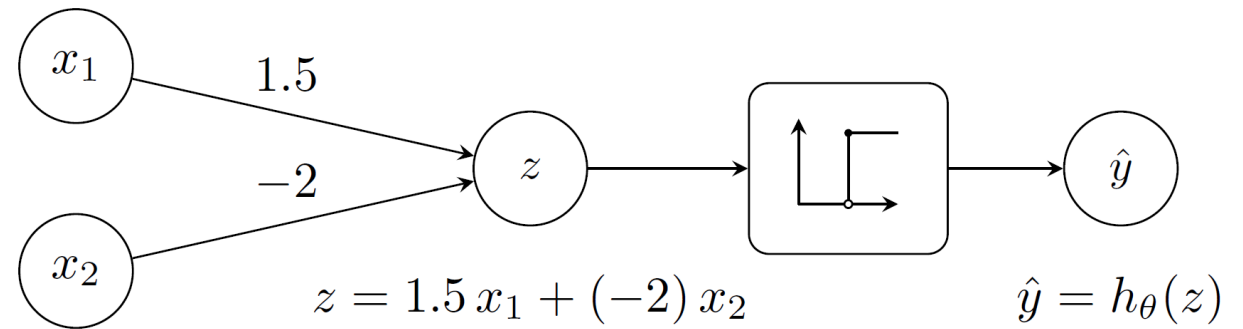
$$\hat{y} = 1$$

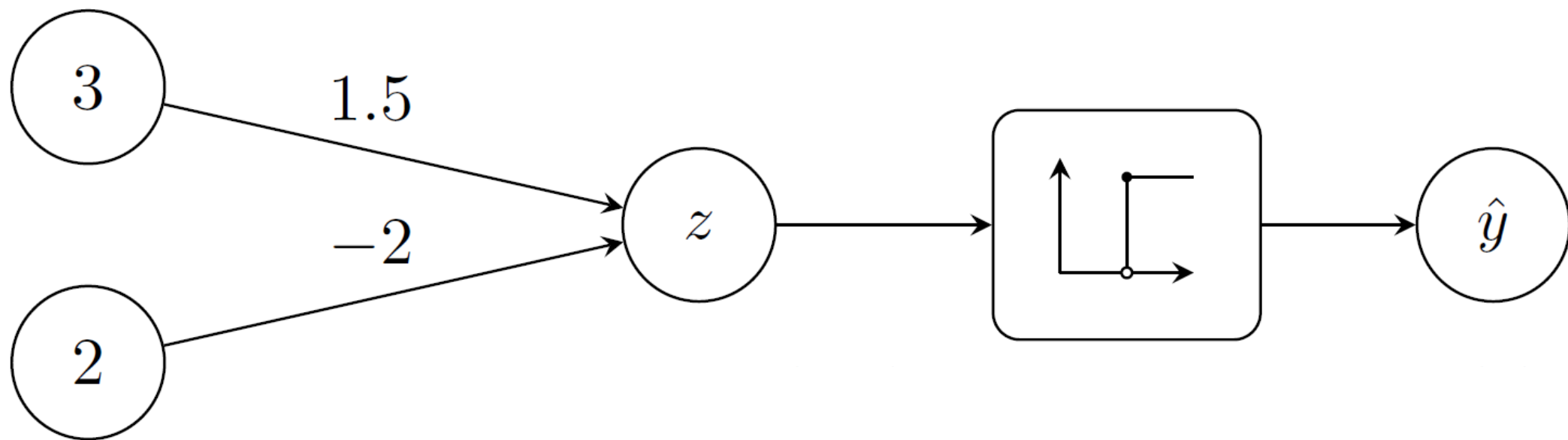
Terskelverdi $\theta: 0$

Læringsrate $\eta: 0.1$

Hva blir de nye verdiene for vektene?

$$\Delta w_i = \eta(y - \hat{y})x_i$$



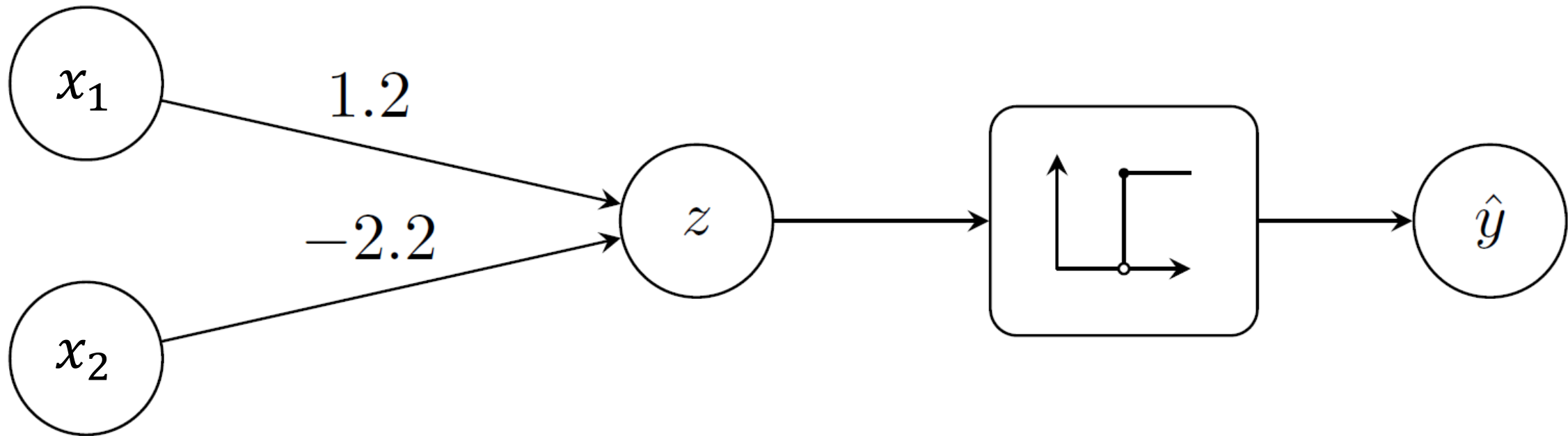


$$\Delta w_i = \eta(y - \hat{y})x_i$$

$$\begin{aligned}\Delta w_1 &= 0.1(0 - 1) \cdot 3 = -0.3, & w_1^{\text{new}} &= 1.5 + (-0.3) = 1.2 \\ \Delta w_2 &= 0.1(0 - 1) \cdot 2 = -0.2, & w_2^{\text{new}} &= -2 + (-0.2) = -2.2\end{aligned}$$

Vårt oppdaterte perseptron

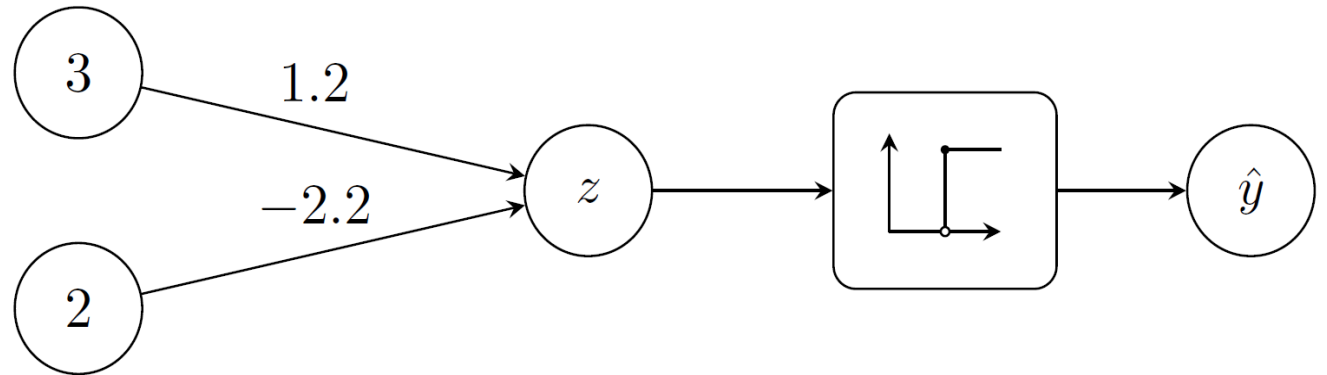
- Vektene endret seg litt etter læring
- **Har perseptronet blitt bedre til å gjenkjenne hunder? Hvordan kan vi teste det?**

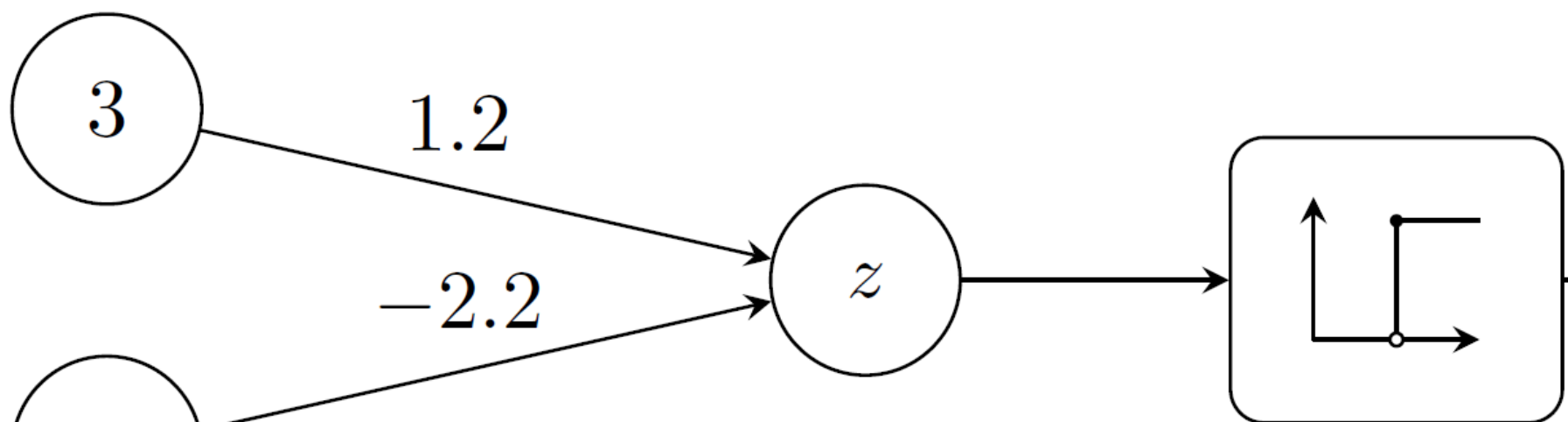


**Vi prøver å sende inn samme inndata
på nytt!**

$$x_1 = 3, x_2 = 2$$

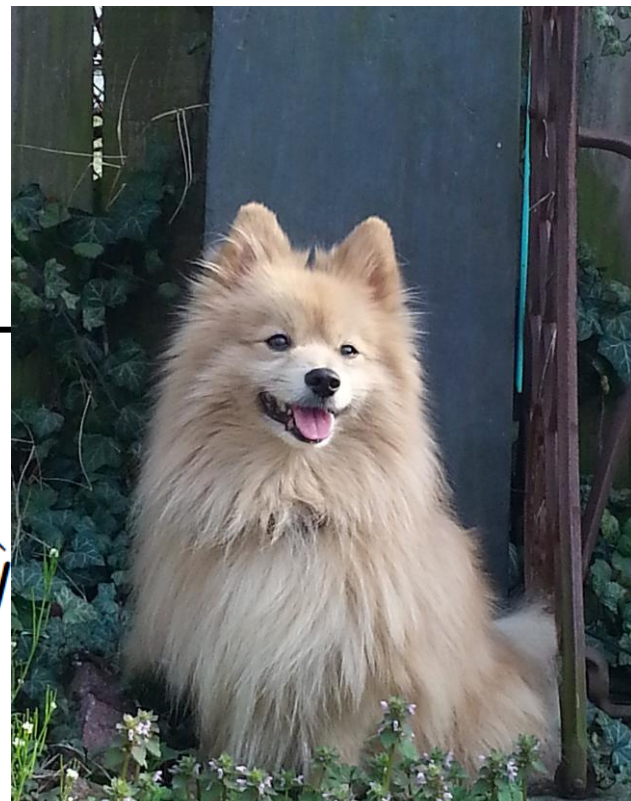
Hva får vi ut nå?
(0 betyr hund, 1 betyr katt)





$$\begin{aligned} z &= 1.2 \cdot 3 + (-2.2) \cdot 2 \\ &= 3.6 - 4.4 = -0.8 \end{aligned}$$

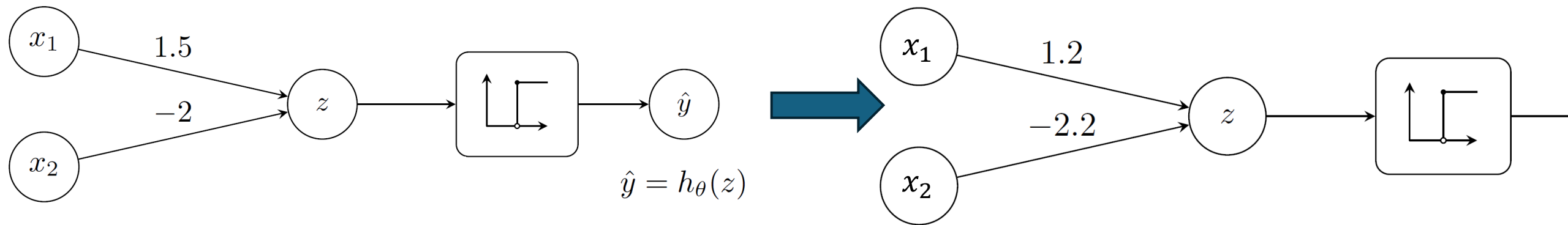
$\hat{y}$



0

Læring fra data

- Perseptronet er mye enklere enn moderne nevrale nettverk
- Men det illustrerer prinsippet: Vi **justerer vekter/parametere ved å se på eksempler**, og blir etter hvert bedre til å **gi svar som stemmer med eksemplene**



Perseptron-læringsregelen

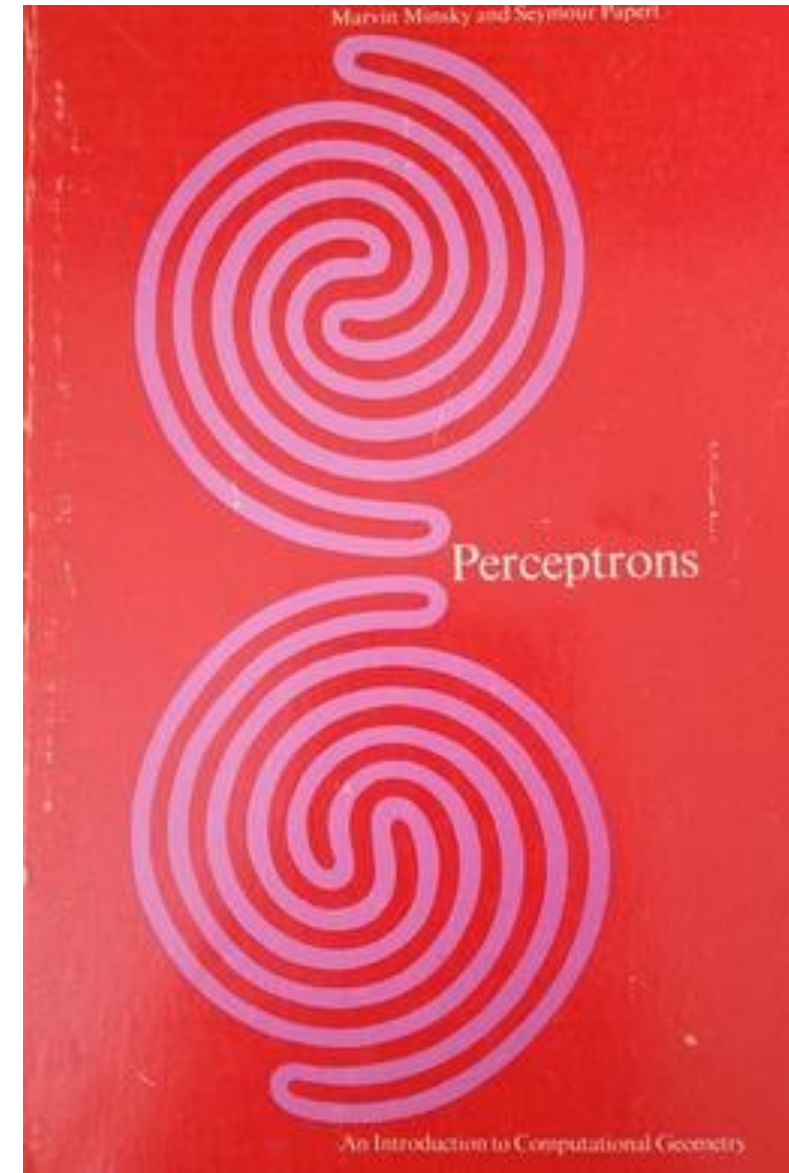
$$(\Delta w_i = \eta(y - \hat{y})x_i)$$

Regelen **garanterer konvergens** til **riktig klassifisering på treningssettet** hvis:

- **Treningsdataene er lineært separerbare, og**
- **Læringsraten er lav nok**

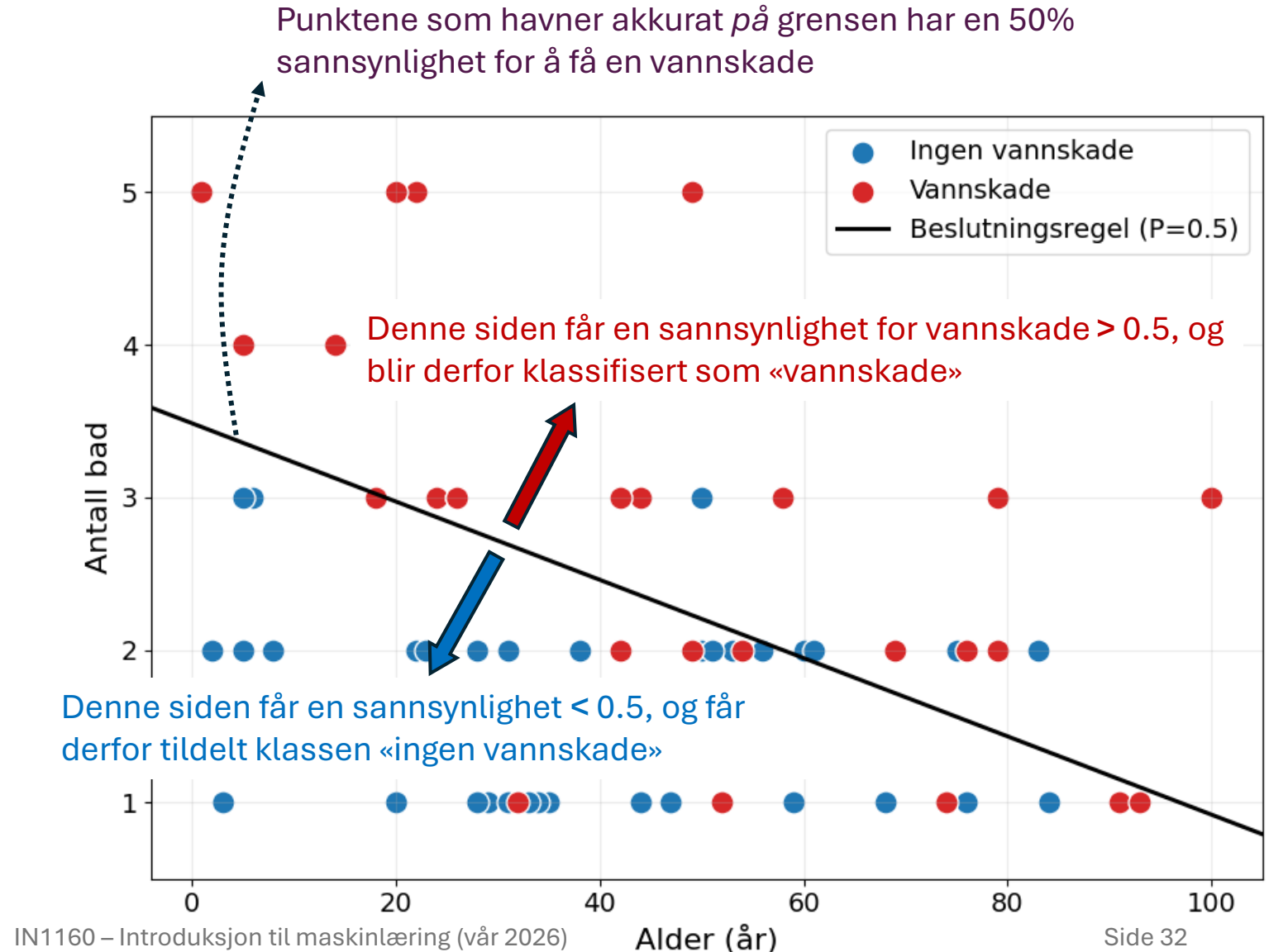
En begrensning ved perseptronet

- Marvin Minsky og Seymour Papert gav i 1969 ut denne boka, der de beviste **en fundamental begrensning ved perseptronet**
- Perseptronet kan kun løse problemer **som er lineært separable**
- De fleste praktiske problemer er dermed for komplekse til å løses av et perseptron



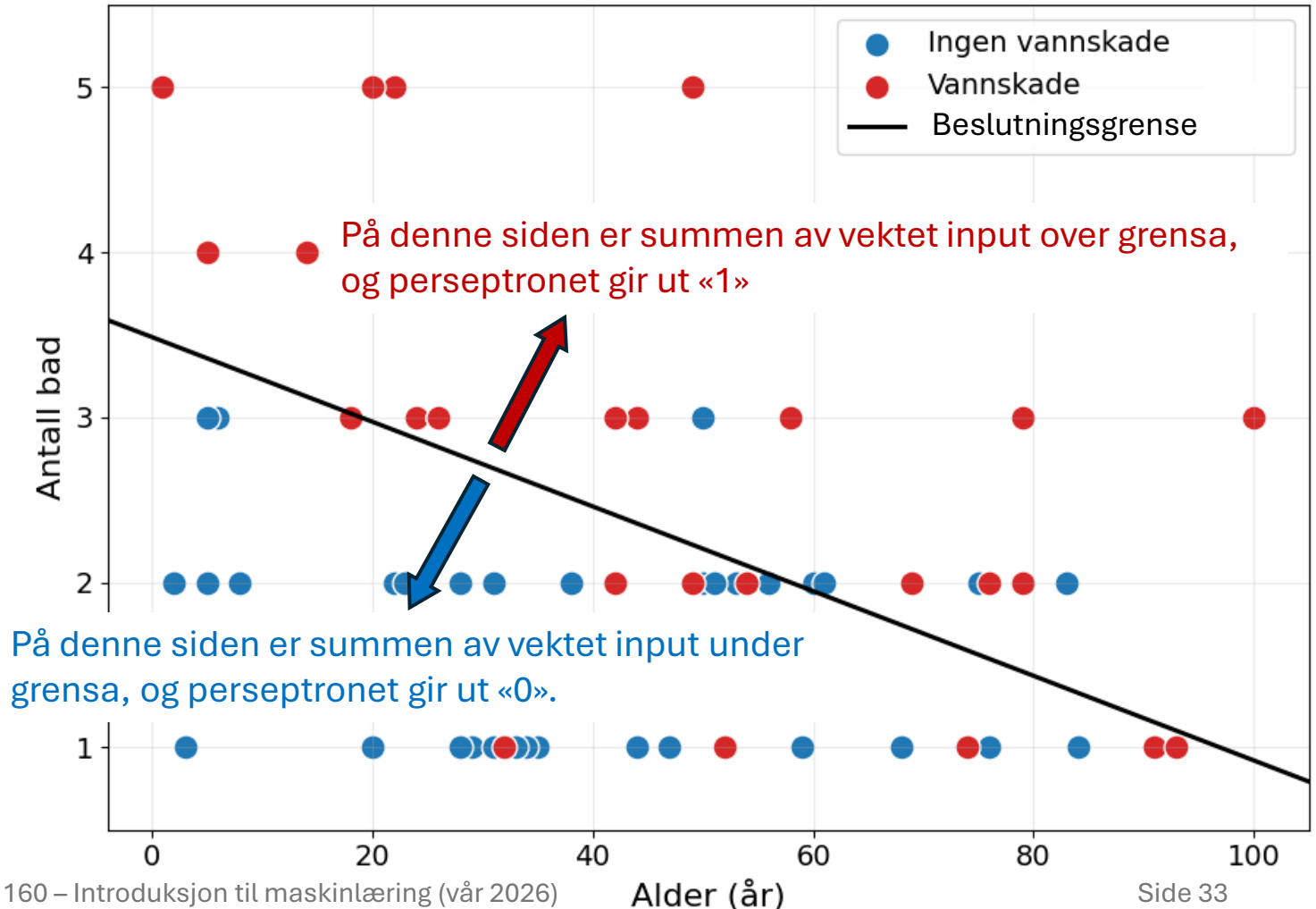
Påminner: Beslutningsgrense fra logistisk regresjon

- Vi kan tegne en såkalt **beslutningsgrense** (eng. *decision boundary*) for å se hva modellen gjør
- Beslutningsgrensen viser hvordan modellen splitter inputtrekkrommet i to:
 - en del hvor $P(c_1|\mathbf{x}) > P(c_2|\mathbf{x})$
 - en del hvor $P(c_1|\mathbf{x}) < P(c_2|\mathbf{x})$
- **Hva er forskjellen på denne og beslutningsgrensen i et perseptron?**



Beslutningsgrense for perseptron

- Beslutningsgrensen er strengere – ingen sannsynlighetsverdier mellom 0 og 1:
 - en del hvor $\hat{y} = 1$
 - en del hvor $\hat{y} = 0$
- Perseptronet gir oss ikke noen sannsynlighetsverdier, som logistisk regresjon gjorde. Det velger bare en klasse.



Beslutningsgrensen flytter seg under trening

<https://jasonhand.github.io/perceptron/>



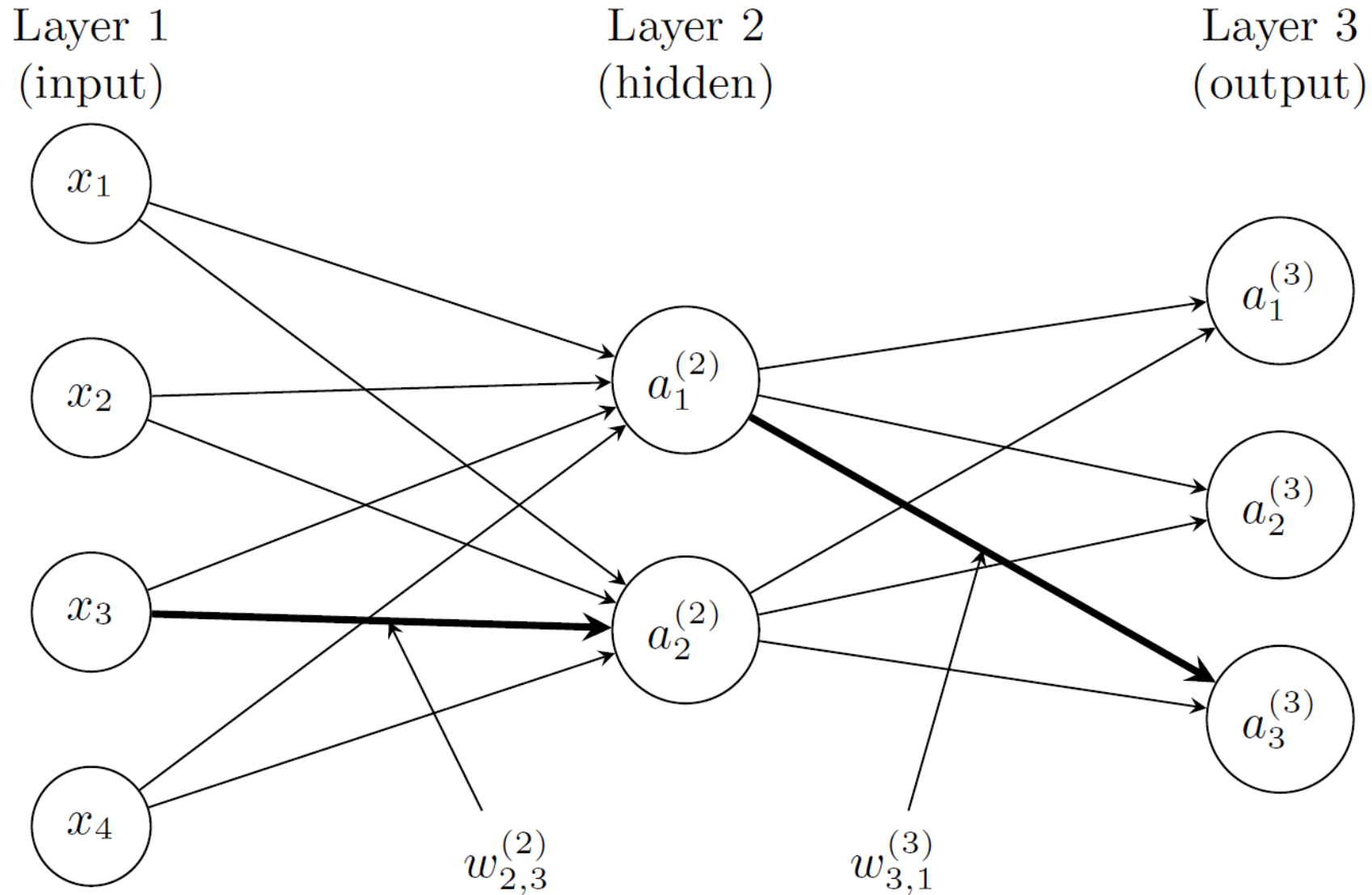
3. Flerlags Nevrale Nettverk

En del lærebøker i ML kaller disse “Flerlags Perseptroner” (MultiLayer Perceptrons, MLP). Men et vanligere ord for dem er rett og slett Nevrale Nettverk.

Hvorfor flere lag?

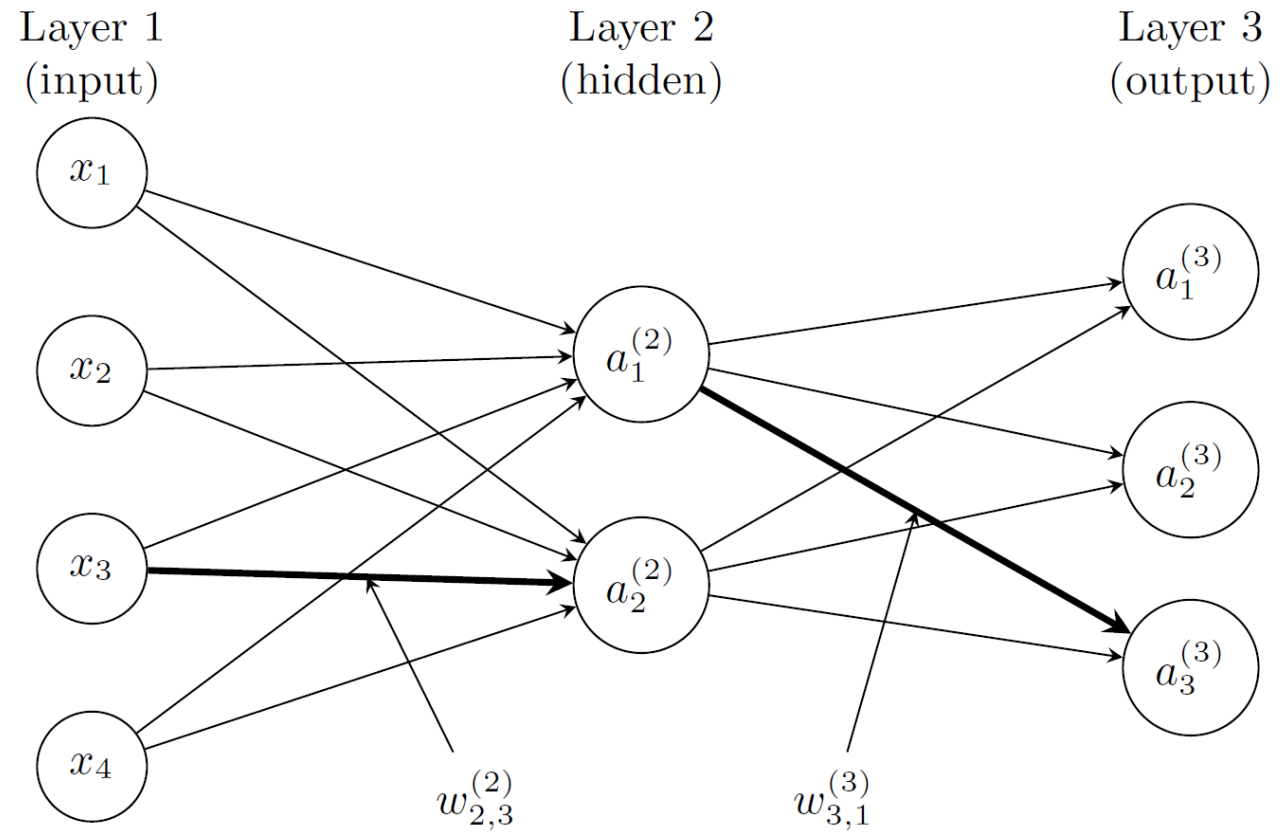
- Vi så at **ettlags perseptroner** kun kan lage **lineære beslutningsgrenser**
- Legger vi til **flere lag**, kan vi kombinere flere segmenter, og få **mer komplekse grenser**.
- Flerlags nevrale nettverk **kan i teorien lære seg en hvilken som helst mapping** fra inndata til utdata ([universell approksimering](#))
 - Men dette krever at vi finner de riktige verdiene til alle vekter – det kan være vanskelig
- Demo: [Neural Network Playground](https://playground.tensorflow.org): playground.tensorflow.org

Flerlags Nevralt Nettverk



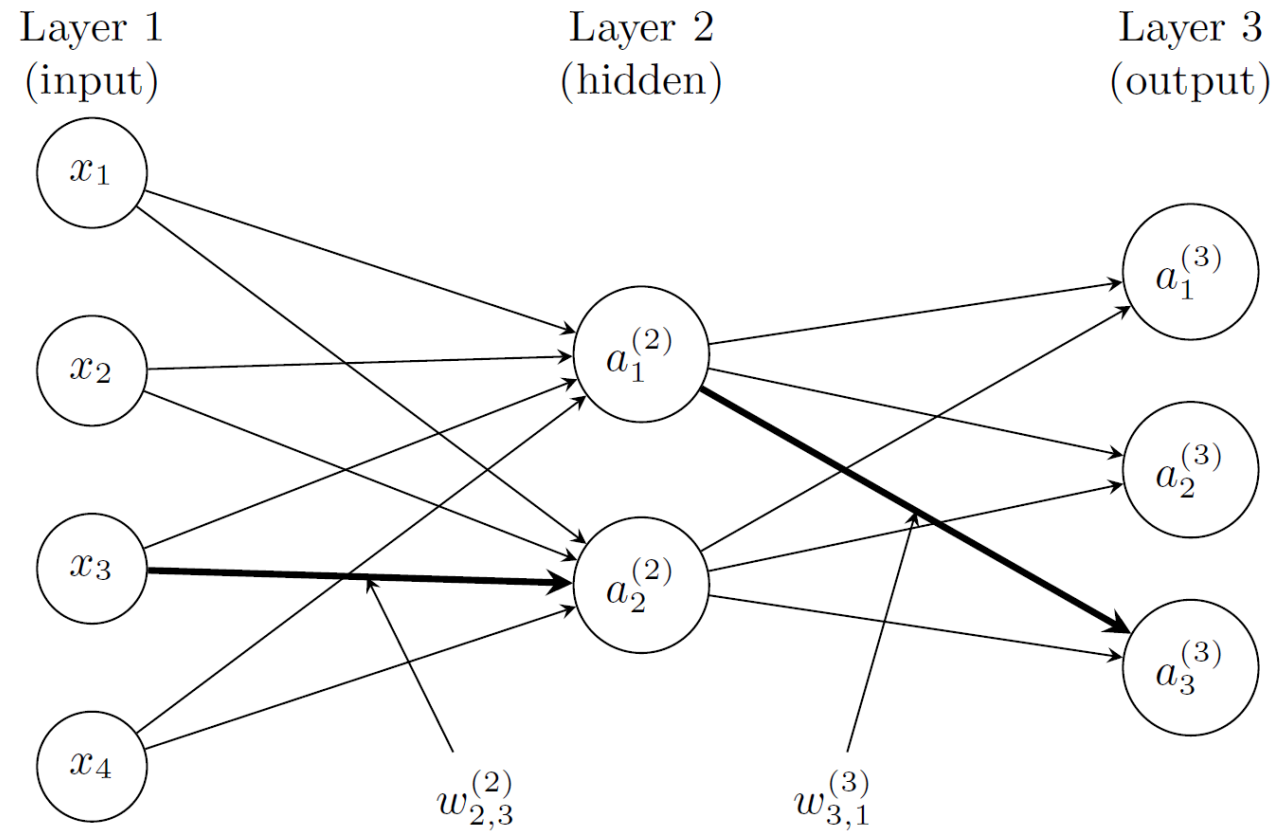
Flerlags Nevralt Nettverk

- De ekstra lagene hjelper å dekomponere problemet, og å lære mer komplekse beslutningsgrenser
- Lagene mellom input og output kalles ofte «skjulte» (hidden) lag
- Det nevrale nettverket har 2 faser:
 - 1) **Forover-fasen:** Beregner utverdier basert på inndata
 - 2) **Bakover-fasen:** Oppdaterer vektor basert på ulikhet mellom ut-verdi og fasit



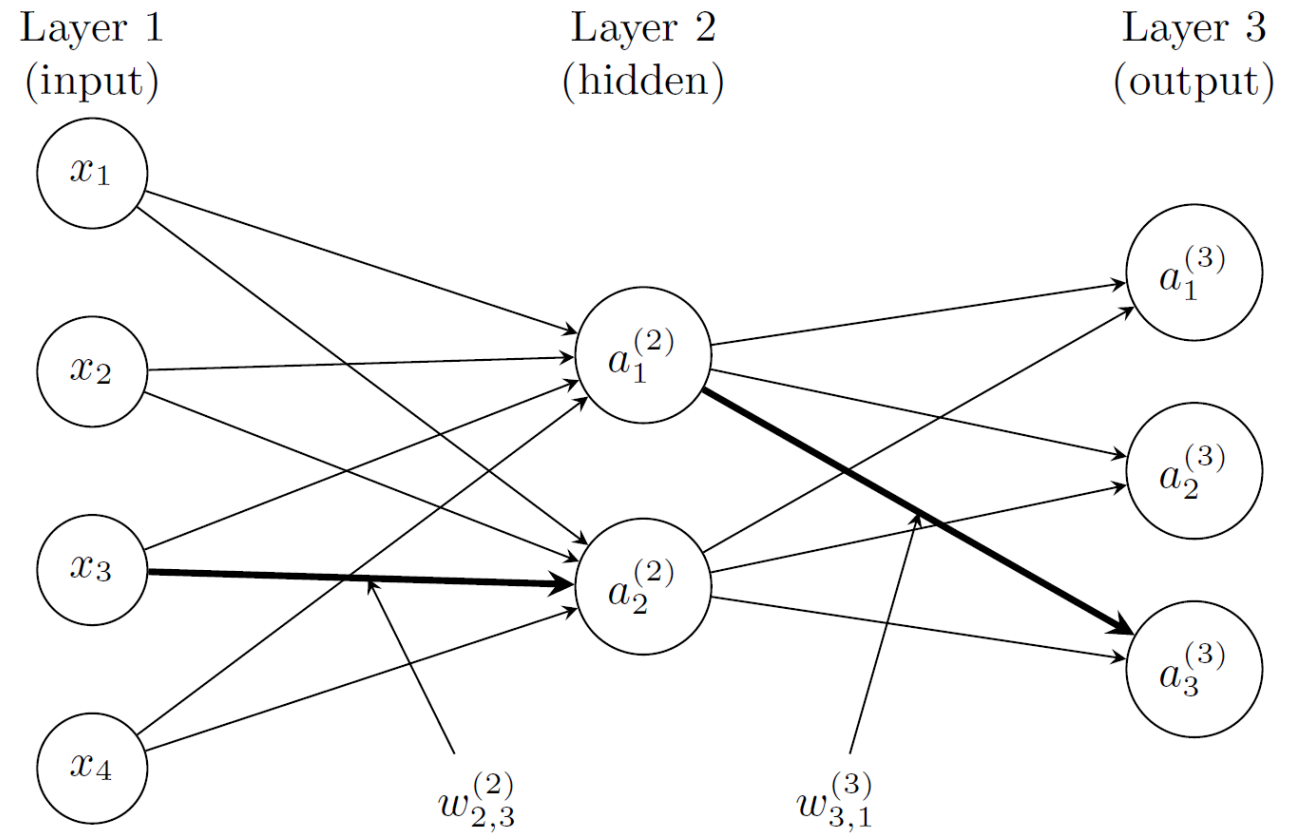
Forover og bakover

- **Bakover-fasen brukes kun under trening.** Oppdatering av vekter skjer der.
- **Foroverfasen** brukes både i **trening** og når vi bruker nettverket etter trening (når vi lager **nye prediksjoner**)

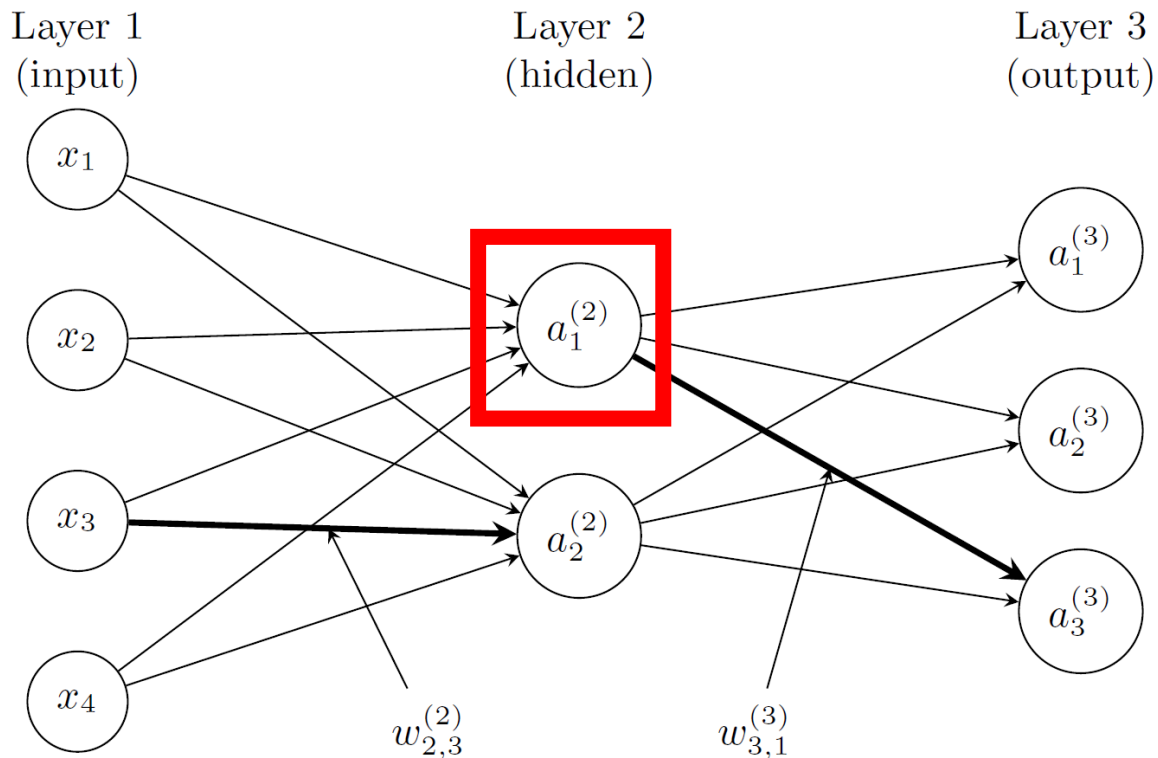


Forover-fasen

- Likner på hvordan vi summerte inputs og anvendte aktiveringsfunksjonen i ettlags-perseptronet. 2 viktige forskjeller:
 - 1) Ut-verdiene sendes som innverdier til neste lag
 - 2) En mer kompleks **aktiveringsfunksjon** (for eksempel en **sigmoid**)



Hva skjer i hvert enkelt nevron?

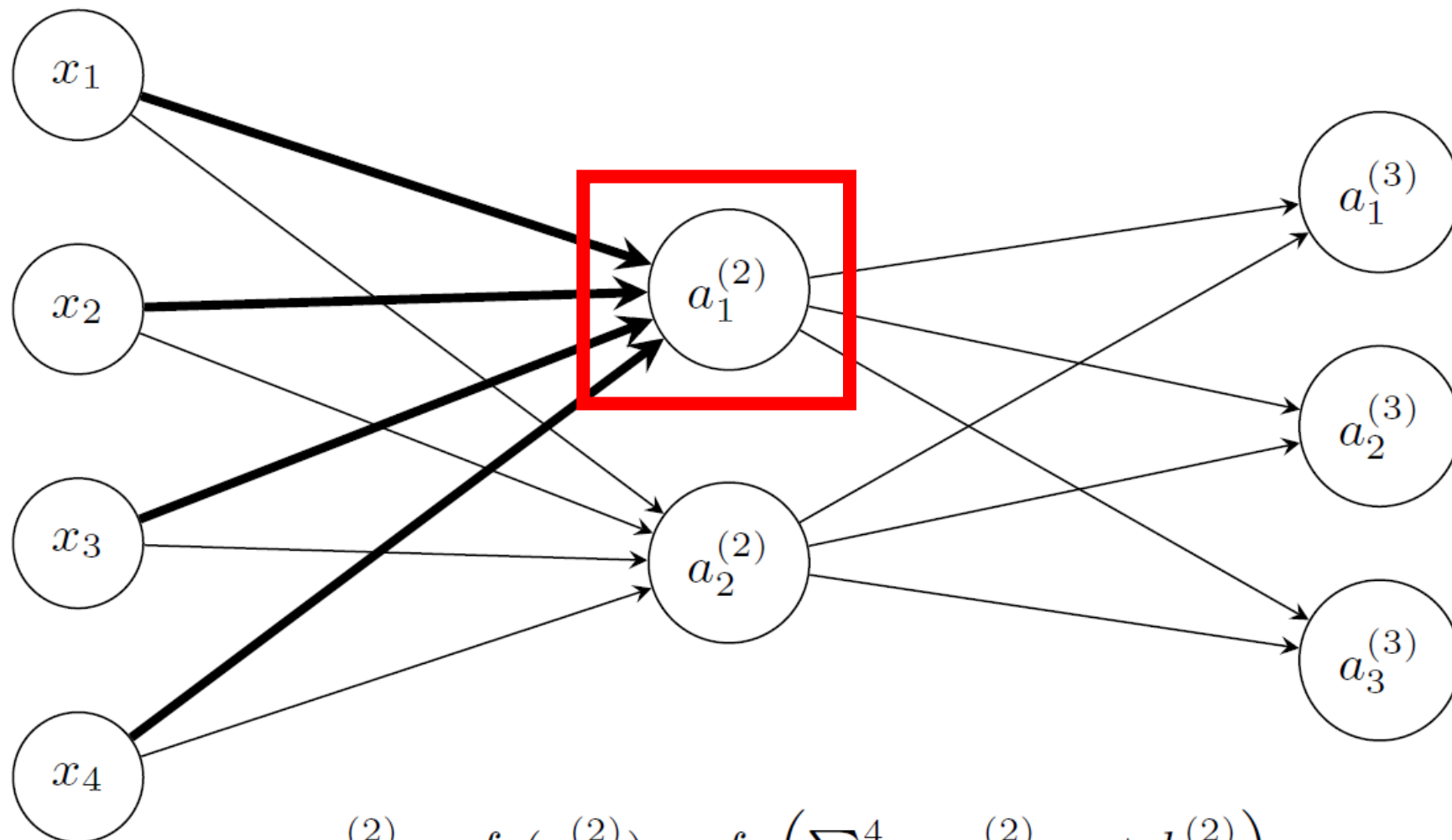


- Som perseptronet **summerer** nevronet den **vektede innverdien**
- Så **sendes denne summen inn i aktiveringsfunksjonen**
- Den endelige aktivitetsverdien $a_1^{(2)}$ sendes videre til neste lag i nettverket.

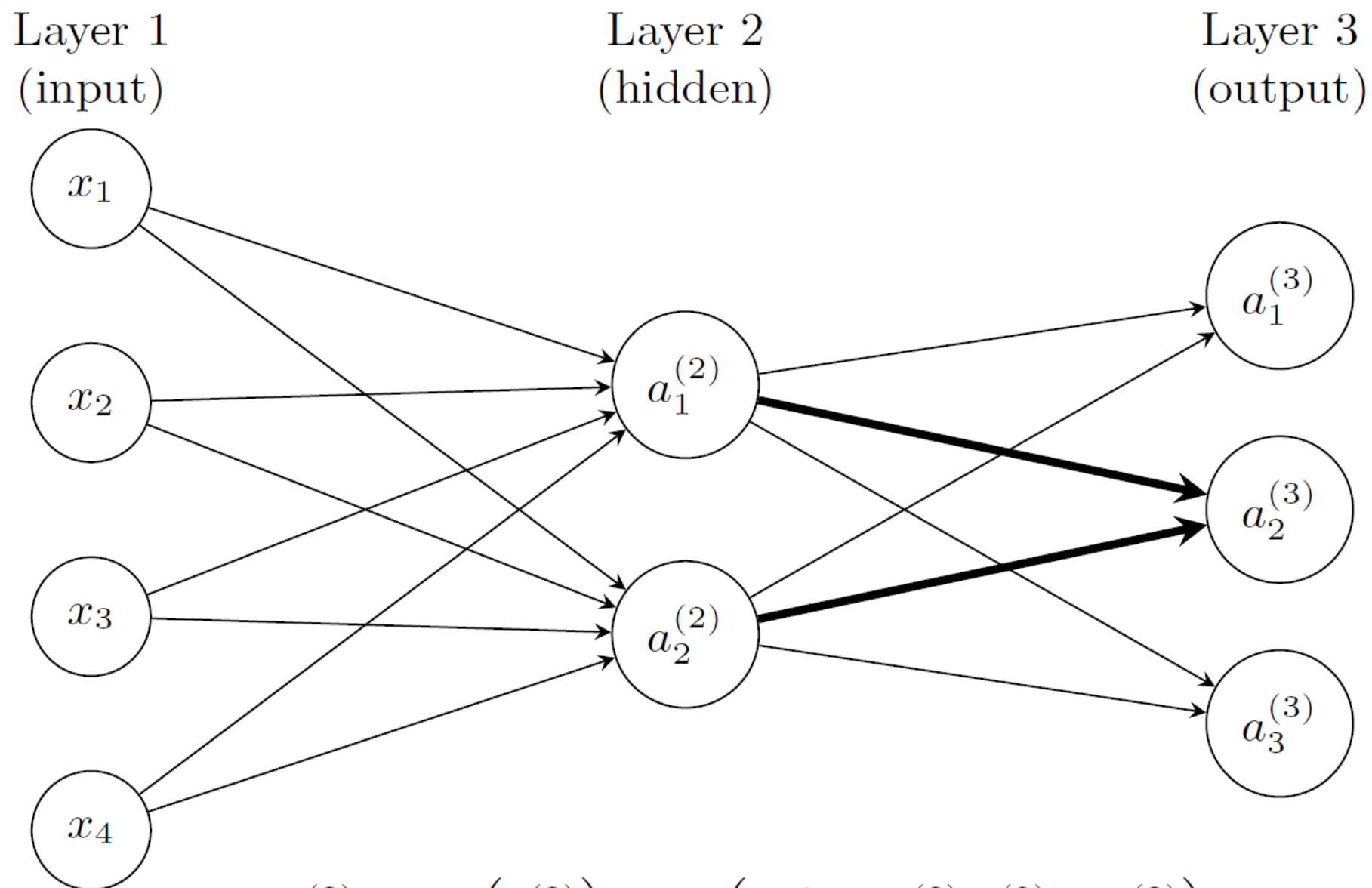
Layer 1
(input)

Layer 2
(hidden)

Layer 3
(output)



$$\begin{aligned} a_1^{(2)} &= f_2(z_1^{(2)}) = f_2 \left(\sum_{i=1}^4 w_{1,i}^{(2)} x_i + b_1^{(2)} \right) \\ &= f_2 \left(x_1 w_{1,1}^{(2)} + x_2 w_{1,2}^{(2)} + x_3 w_{1,3}^{(2)} + x_4 w_{1,4}^{(2)} + b_1^{(2)} \right) \end{aligned}$$



$$\begin{aligned}
 a_2^{(3)} &= f_3 \left(z_2^{(3)} \right) = f_3 \left(\sum_{j=1}^2 w_{2,k}^{(3)} a_k^{(2)} + b_2^{(3)} \right) \\
 &= f_3 \left(w_{2,1}^{(3)} a_1^{(2)} + w_{2,2}^{(3)} a_2^{(2)} + b_2^{(3)} \right)
 \end{aligned}$$

Forover-fasen oppsummert

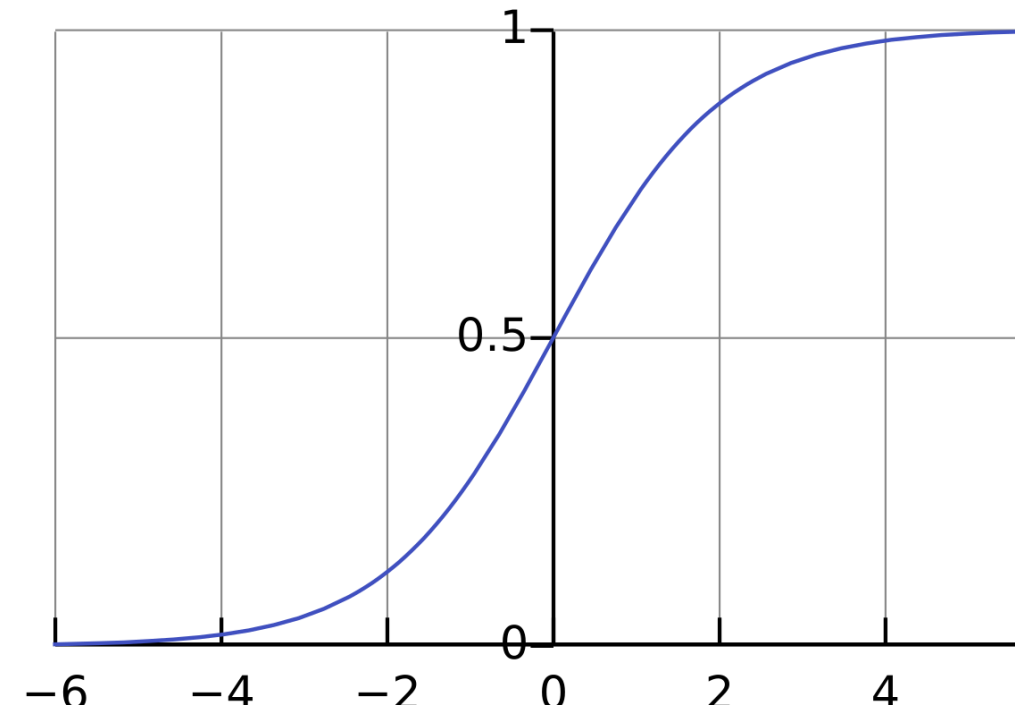
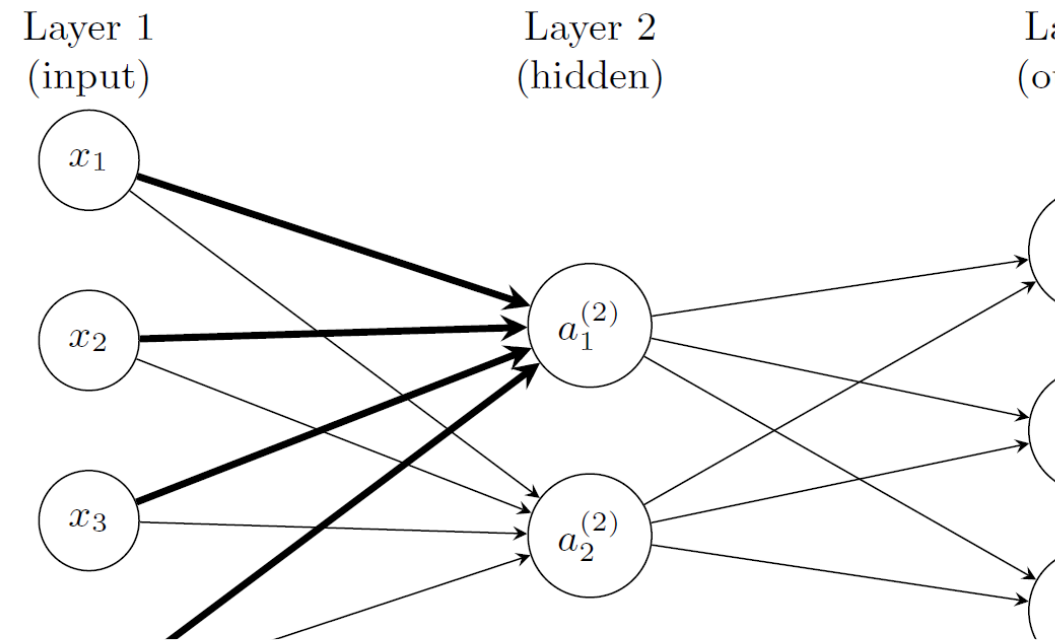
- Hvert nevron gjør følgende:

1) **Summer vektete inndata**

$$z = \left(\sum wx \right) + b$$

2) Send denne summen inn til
aktiveringsfunksjonen $a=f(z)$

- Resultatet $a=f(z)$ **sendes videre til neste nevroner lenger ned i nettverket via vektene** som finnes der
- Til slutt får vi **verdier på ut-nevronene** som er **nettverkets endelige prediksjon** for disse inndataene



Bakover-fasen: Tilbakepropagering (eng: Backpropagation)

- Man lette lenge etter en effektiv måte å trene (justere vektene) flerlags nevrale nettverk på
- For ettlags-perseptroner så vi en grei løsning: $\Delta w_i = \eta(y - \hat{y})x_i$
- For flerlags-nevrale nettt er det mer komplekst. Vi må finne ut **hvor mye «ansvar» hver av koblingene i hvert lag har for feilen** – og hvordan vi justerer den for å gjøre feilen mindre.
- 1986: «Learning representations by **back-propagating errors**» (David E. Rumelhart, Geoffrey E. Hinton & Ronald J. Williams)

Påminner:

Gradientnedstigning

- Lineær regresjon og logistisk regresjon brukte denne teknikken for å lære parametere.

Treningsprosedyren blir altså:

1. Start med tilfeldige verdier for parameterne (**w**, **b**)

Gjenta:

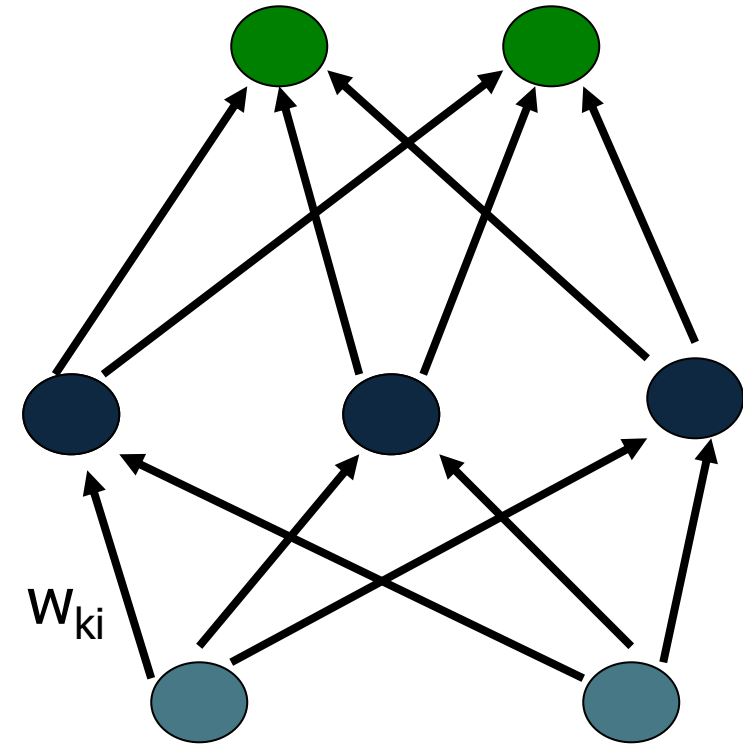
2. Ta en *mini-batch* $\mathcal{B}$ av treningsdata
3. Beregn **hvor mye modellen tar feil** på $\mathcal{B}$
4. Beregn ***gradienten* av dette tapet med hensyn på parameterne**
5. **Juster parameterne et lite steg i retning hvor tapet synker raskest**

... inntil tapet ikke lenger går ned

Gradientnedstigning i nevrale nettverk

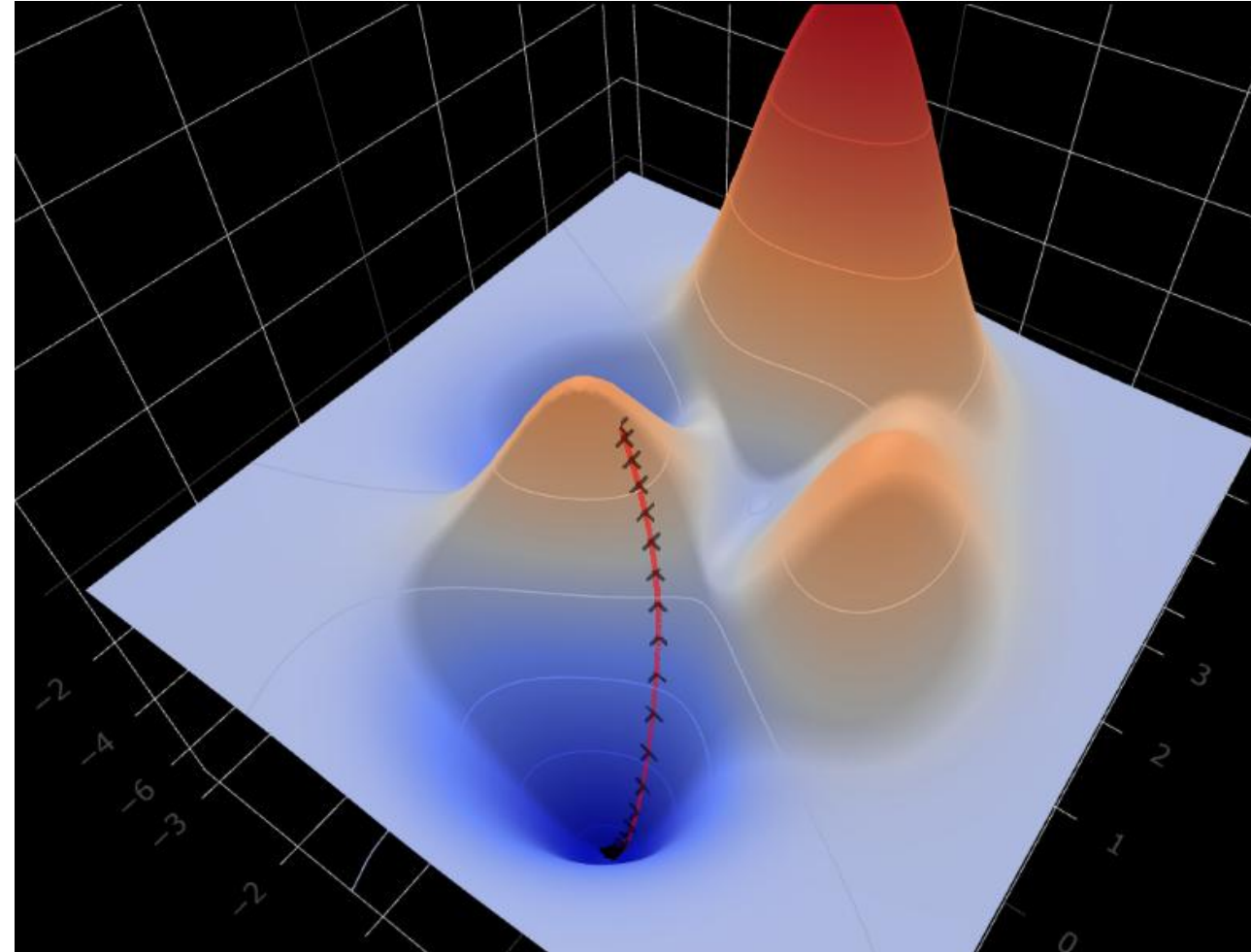
- Vi ønsker å finne ut: For en gitt vekt, w_{ki} , hvordan endres *feilen* E når vekten endres litt?
- Dette kan vi regne ut som den **partiellderiverte** $\frac{\partial E}{\partial w_{ki}}$
- Generelt: **Gradienten** er rett og slett en vektor som inneholder **alle de partiellderiverte**
 - For funksjonen $f(x, y, z)$

er gradienten $\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$



Gradienten illustrert

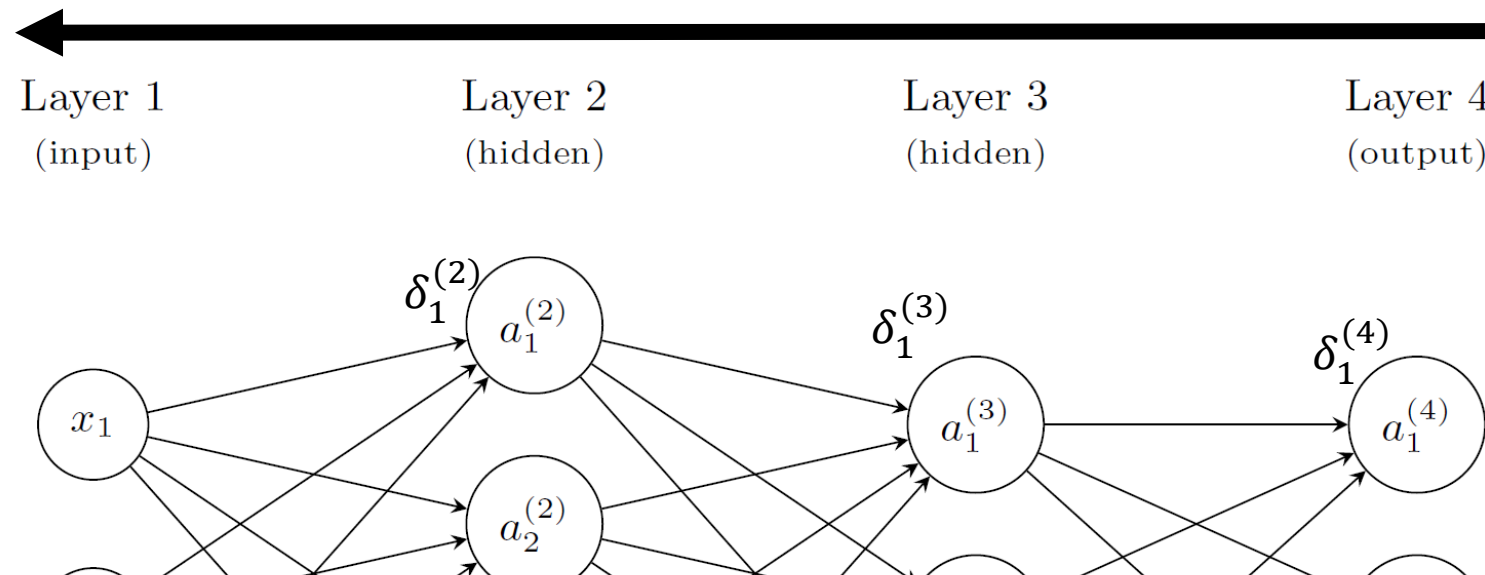
- Vi skal se en demo i 2 dimensjoner (X og Y)
- Vi kan tenke oss at **de to er to vekter i et nevralt nettverk** mens **Z-aksen viser feilen** nettverket vil gjøre for akkurat denne verdien av de to vektene
- Gradienten **viser veien mot å minimere feilen**
- <https://blog.skz.dev/gradient-descent>



Tilbakepropagering på høyt nivå (1)

Vi ønsker å finne alle de partiellderiverte $\frac{\partial E}{\partial w_{ki}}$

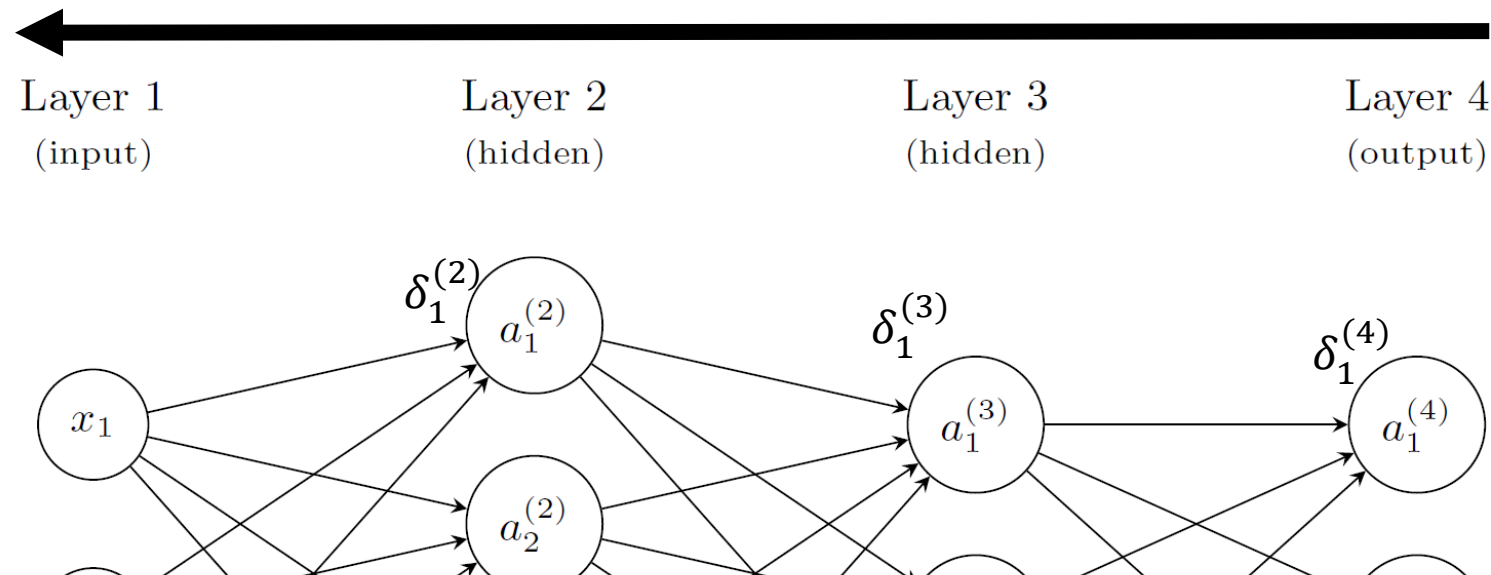
For **vekt** i **interne lag** i nettverket (lag 2 og 3 under) krever denne beregningen at vi regner ut info om **partiellderiverte nærmere utgangen** i nettverket først. Denne infoen «**propagerer tilbake**».



Tilbakepropagering på høyt nivå (2)

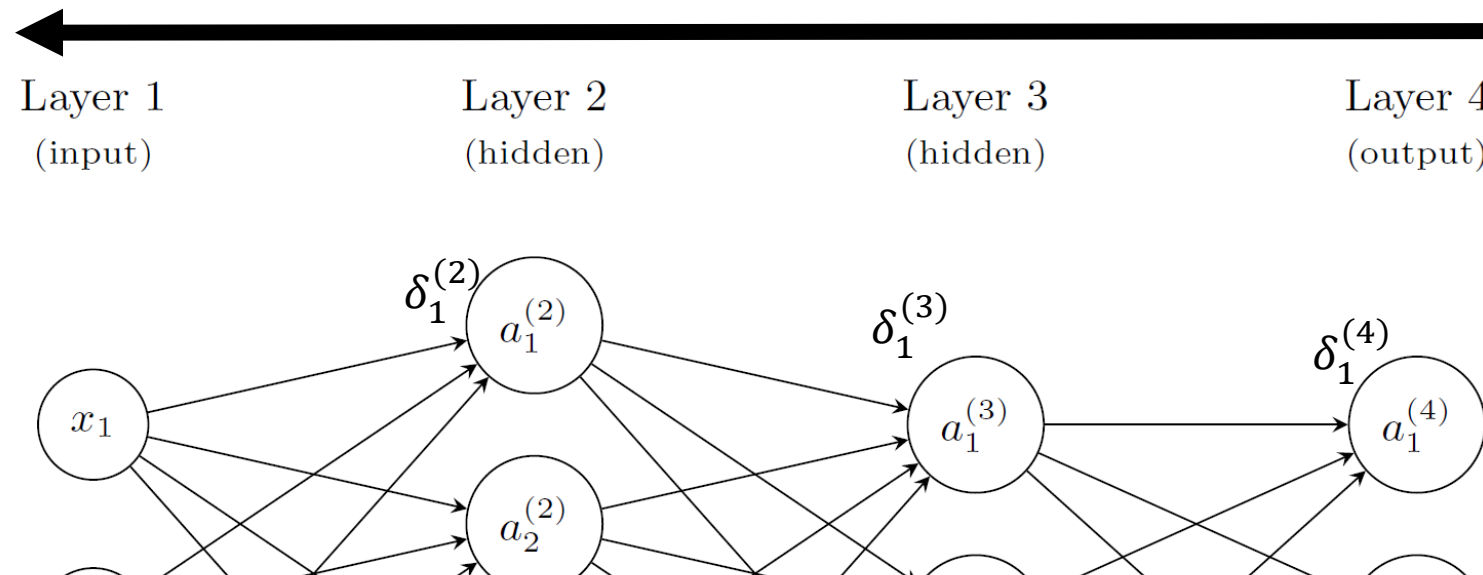
Vi ønsker å finne alle de partiellderiverte $\frac{\partial E}{\partial w_{ki}}$

Denne beregningen krever også at vi beregner **deriverte av aktiveringsfunksjoner** – med hensyn på input. Derfor viktig at **aktiveringsfunksjoner i nevrale nett er deriverbare!**

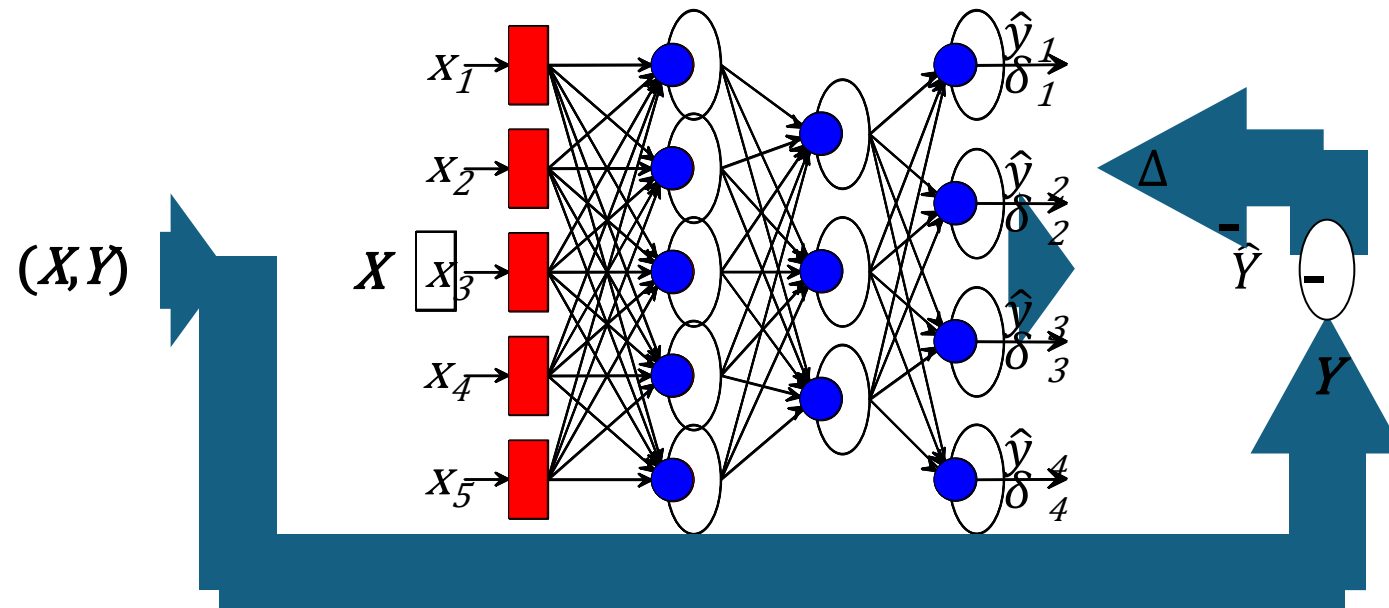


Tilbakepropagering på høyt nivå (3)

- 1) **Beregn hvor stor feil** ($\delta_i^{(j)}$) ut-nevront/nevronene gjør. For eksempel Mean Squared Error (MSE)
- 2) **Informasjon om denne feilen** sendes /propagerer tilbake i nettverket gjennom hvert skjulte lag ($\delta_i^{(k)}$)
- 3) **Vektene i lag k oppdateres**, ved bruk av dette **feil-signalet** ($\delta_i^{(k)}$)

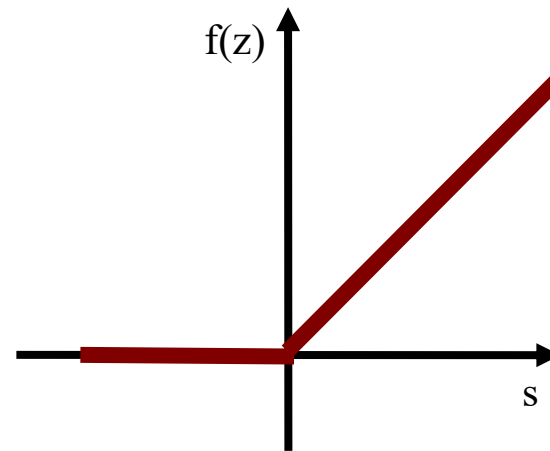
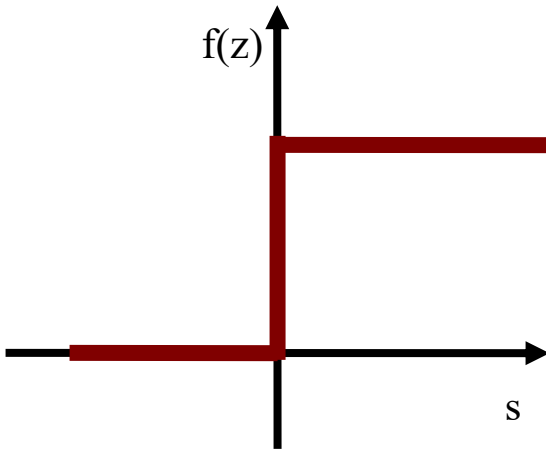


Tilbakepropagering



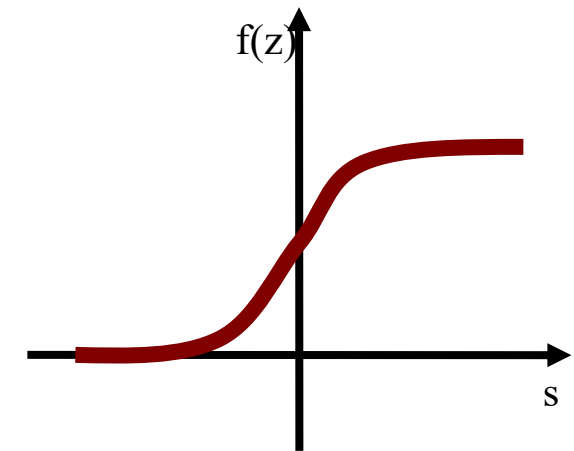
Noen vanlige aktiveringsfunksjoner

Terskel – brukes i ettlags-perseptronet, men ikke vanlig i moderne Nevrale Nett



ReLU (Rectified Linear Unit)

sigmoid



ReLU og sigmoid: Gir noe av samme “binære” effekten som terskel-verdien, men har den fordel at de er **deriverbare**

Tilbakepropagering - pensum

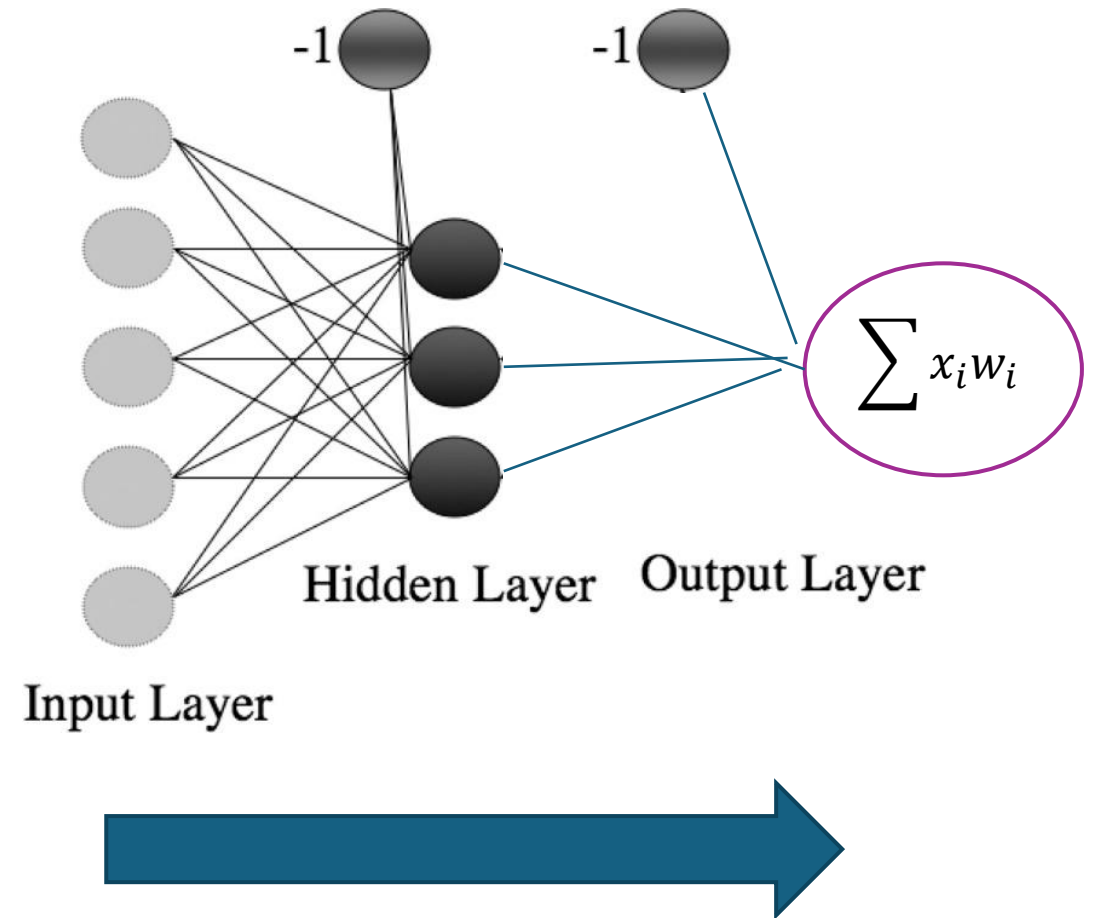
- Vi forventer ikke at dere kjenner matematikken bak tilbakepropagering i dette kurset
- Men: At dere kjenner intuisjonen av **hvordan informasjon flyter forover og bakover** i nevralt nettverk, og at dere har en **høynivå forståelse for treningsprosessen** er pensum
- **Beregningene i forover-fasen** er enklere, disse er pensum å kunne.
- Dypere dykk ned i matematikken kommer i senere ML-kurs på IFI!

Nevrale Nettverk – ulike typer problemer

- Nevrale Nettverk kan brukes både til **klassifikasjon og regresjon**.
- Det som skiller de to er hvordan vi beregner **ut-data og feilen (tapsfunksjonen)** i nettverket. Vi skal se noen eksempler
 - 1) Regresjon
 - 2) Binær klassifikasjon
 - 3) Flerklasse-klassifikasjon

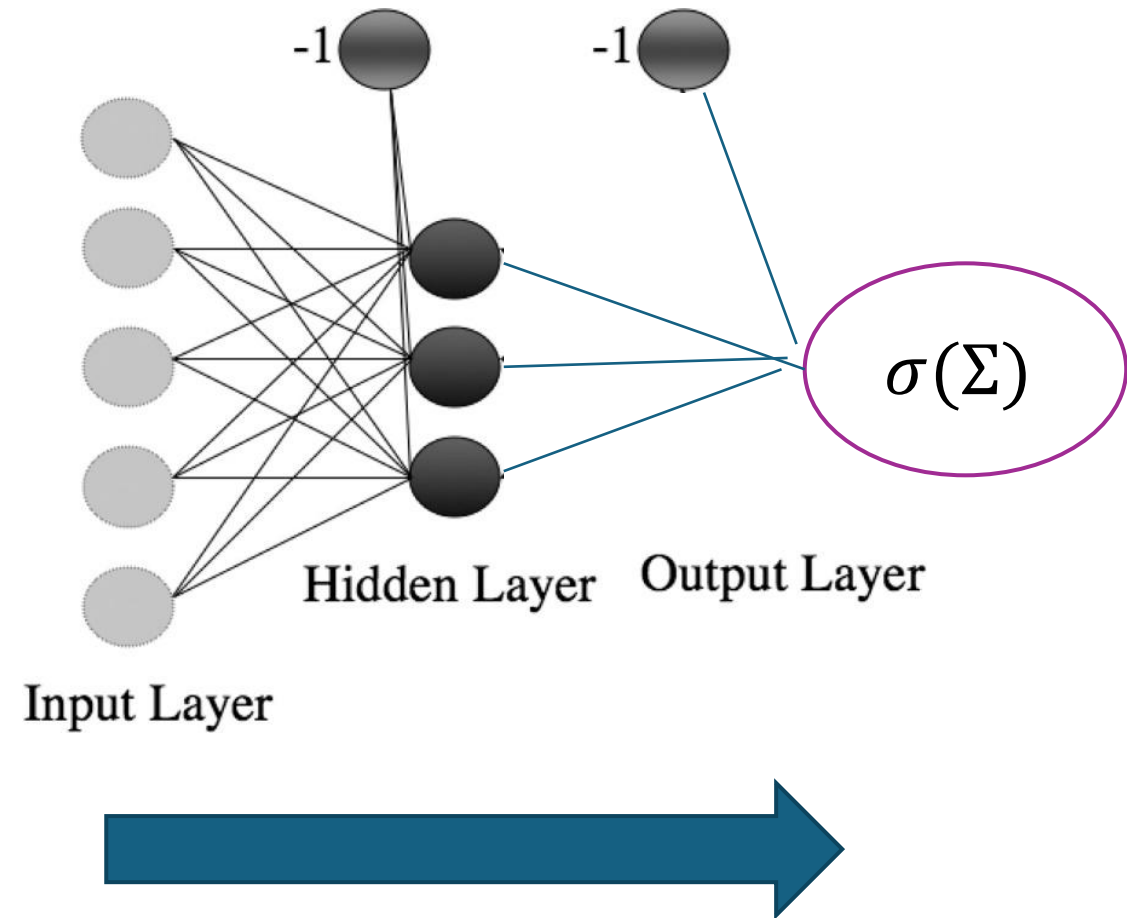
1. Regresjon

- Kun én ut-node
- Ingen aktiveringsfunksjon i ut-laget
 - Eller vi kan tenke oss at den er identitetsfunksjonen: $f(s) = s$
 - De skjulte lagene har aktiveringsfunksjoner som vanlig
- Siden vi har flere lag kan vi modellere ikke-lineære regresjonsfunksjoner
- **Tapsfunksjon kan være MSE,** som for lineær regresjon



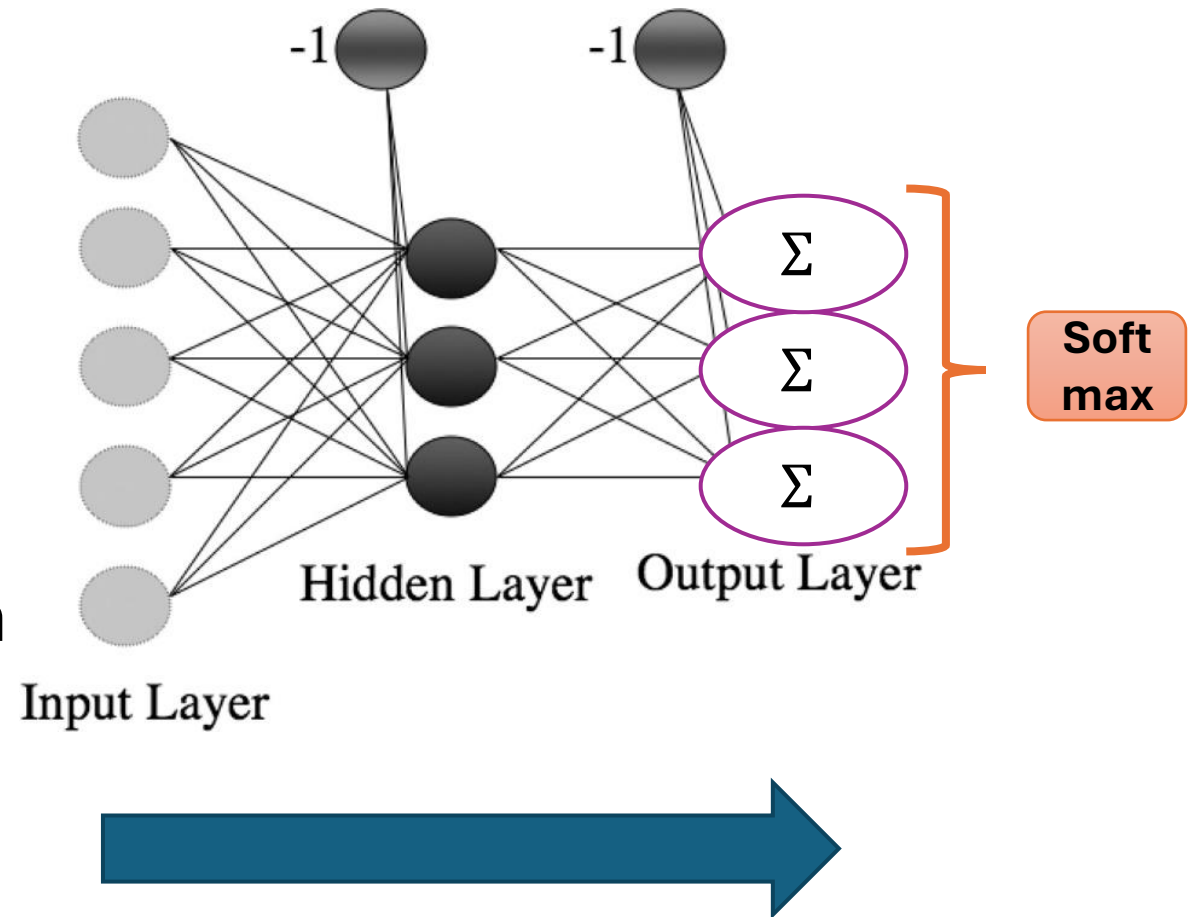
2. Binær klassifikasjon

- Én enkelt ut-node
- Sigmoid aktiveringsfunksjon i utlaget
 - Får en effekt som likner logistisk regresjon
- Vi kan få **ikke-lineære beslutningsgrenser**
- Tapsfunksjon: **Kryssentropi (gjennomgått sist uke)**



3. Flerklasse-klassifikasjon

- Likner **multinomial logistisk regresjon**: Hver ut-node representerer en av klassene
- Vi tolker ut-verdiene som sannsynlighetsverdier for de ulike klassene.
- F.eks. $[0.2, 0.2, \mathbf{0.6}]$ tolker vi som at **klasse 3** er mest sannsynlig
- Tapsfunksjon: Kryssentropi



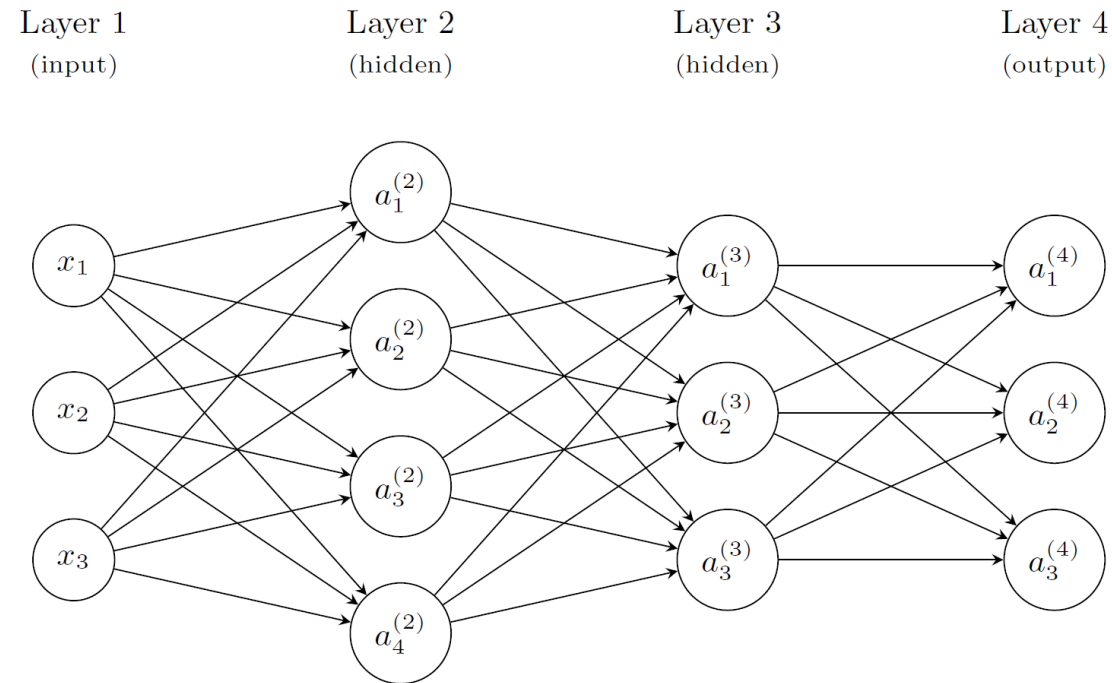


4. Noen praktiske detaljer

Praktisk implementasjon og testing av nevrale nettverk får dere også prøvd dere på i Oblig 3b!

Hvor mange lag og hvor mange noder i hvert?

- Valget involverer ofte **prøving og feiling**, og å se på hva andre har gjort før
- Bruk et **uavhengig valideringssett** når du sammenlikner ulike oppsett
- Velg det oppsettet som fungerte best på valideringssettet
- Dette kalles **hyperparameterjustering**
 - **Vektene i nettverket kalles ofte parametere**, innstillingen til selve læringsalgoritmen kalles derfor hyperparametere



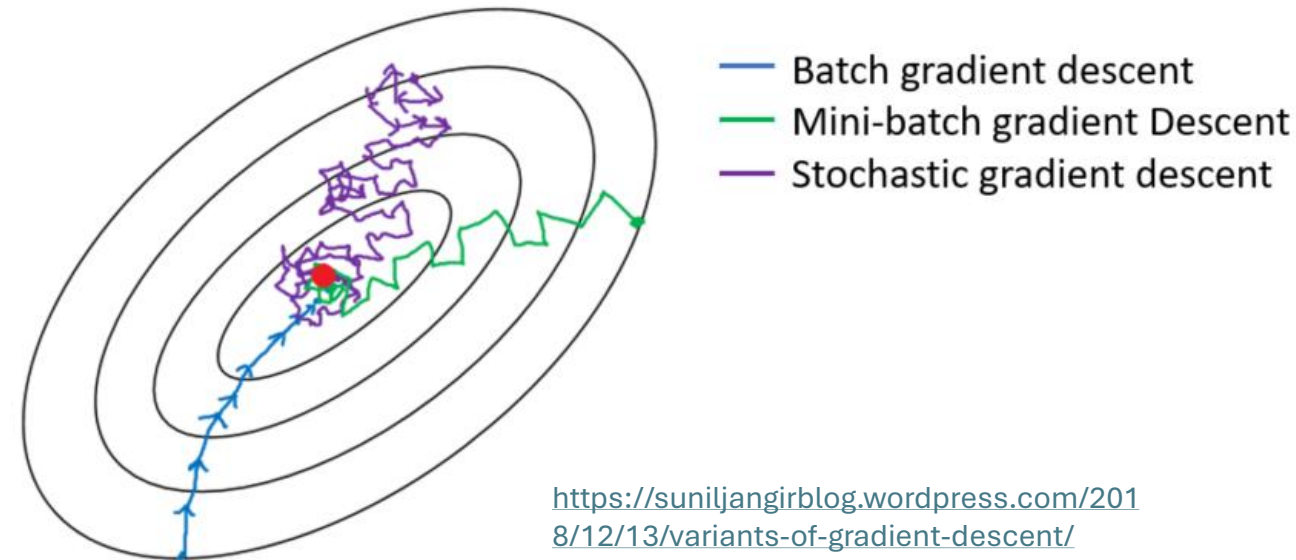
«ML i praksis»-forelesningen og Oblig 3a

Når skal vi oppdatere vektene?

- Vi ser hvert datapunkt flere ganger, og oppdaterer vektene til vi er ferdig med treningen
- Men akkurat når vi oppdaterer vektene varierer:
 - Oppdatering **etter hvert enkelt** datapunkt (**Stokastisk Gradientnedstigning**)
 - Oppdatering **kun når vi har sett alle data** – oppdater i retning av gjennomsnittlig feil (**batch-trening**)
 - En **mellomting** (**mini-batch**)

Variasjoner av gradientnedstigning

- Mini-batch trening:
 - Velg en delmengde av datasettet, av en gitt størrelse
 - Beregn tapsverdi for disse dataene
 - Beveg vektene en gang i motsatt retning av gradienten
- Batch trening
 - Bruk hele treningssettet i hver oppdatering
- Stokastisk gradientnedstigning:
 - Beregn tapsverdien for et tilfeldig datapunkt, og beveg vektene i motsatt retning av gradienten



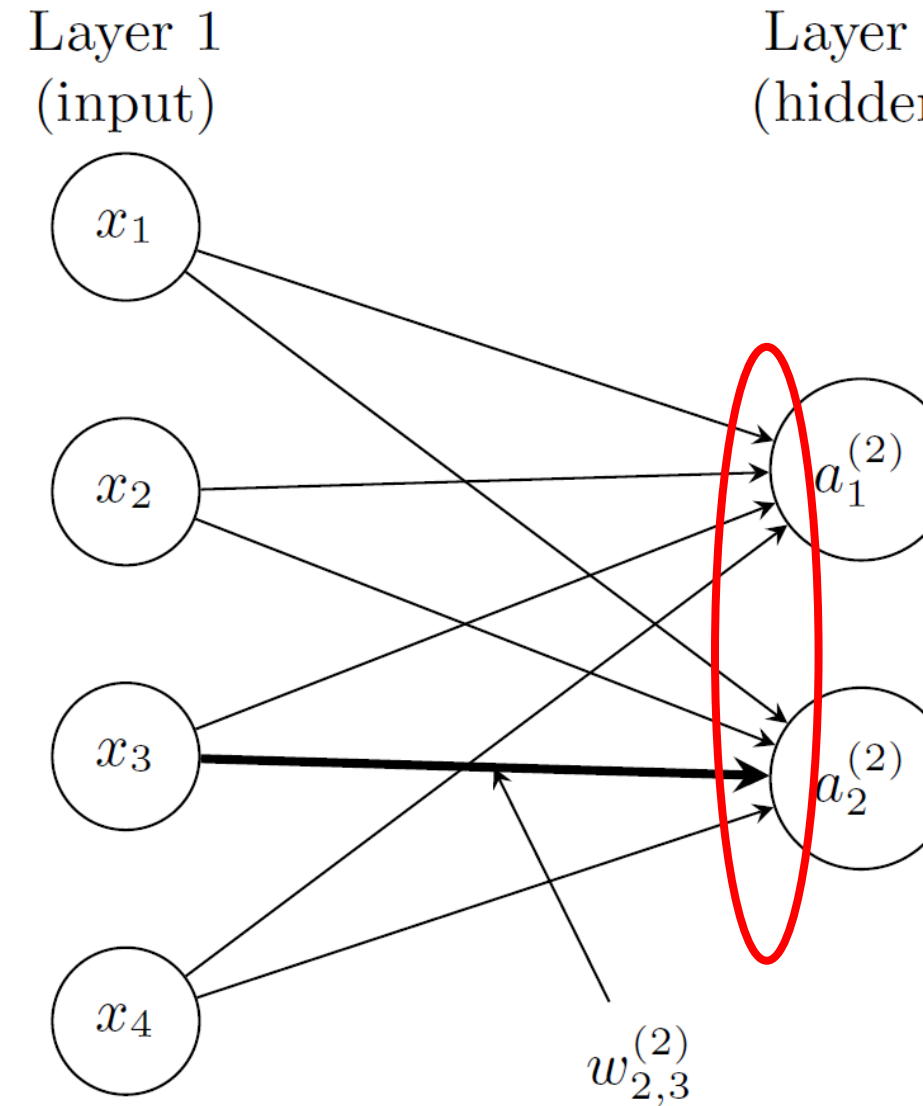
- Mini-batching brukes mest i praksis. Vi kan velge en batch-størrelse tilpasset datamaskinens minne for effektiv beregning

Matriser

- I praksis gjøres alle beregninger med matriser

$$\begin{aligned} W^{(2)}\mathbf{x} + \mathbf{b}^{(2)} &= \begin{bmatrix} w_{1,1}^{(2)} & w_{1,2}^{(2)} & w_{1,3}^{(2)} & w_{1,4}^{(2)} \\ w_{2,1}^{(2)} & w_{2,2}^{(2)} & w_{2,3}^{(2)} & w_{2,4}^{(2)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \end{bmatrix} \\ &= \begin{bmatrix} w_{1,1}^{(2)}x_1 + w_{1,2}^{(2)}x_2 + w_{1,3}^{(2)}x_3 + w_{1,4}^{(2)}x_4 + b_1^{(2)} \\ w_{2,1}^{(2)}x_1 + w_{2,2}^{(2)}x_2 + w_{2,3}^{(2)}x_3 + w_{2,4}^{(2)}x_4 + b_2^{(2)} \end{bmatrix} \end{aligned}$$

- Mer om matriseform og praktisk implementering i oblig 2b.



Oppsummering

- Nevrale nettverk er inspirert av hjernen, men svært forenklet. Fokuserer på modeller av nevroner som enkle beregningsenheter, der læring skjer ved å justere styrken på vektene mellom dem
- Forover-fasen (brukes **under prediksjon og trening**):
 - Summer innkommende aktivering
 - Transformer denne summen med en aktiveringsfunksjon
- Bakover-fasen (brukes **kun under trening**):
 - Beregn partiellderiverte, som viser hvordan hver vekt må justeres for å minimere feilen i tapsfunksjonen
 - Juster alle vekter i nettet, og la feil-informasjonen propagere bakover



Spørsmål?